

Benchmarking Deep Reinforcement Learning for Cartesian Path Tracking of Free-Floating Space Robot Manipulators

PATRICK S. ERMISCH¹, and ERIC GUIFFO KAIGOM¹

¹Department of Computer Science & Engineering, Frankfurt University of Applied Sciences, Frankfurt am Main, Germany
(e-mail: steven.ermisch@stud.fra-uas.de, kaigom@fra-uas.de)

Corresponding author: Eric Guiffo Kaigom (e-mail: kaigom@fra-uas.de).

ABSTRACT Free-floating space robots exhibit a strong dynamic coupling between manipulator motion and base attitude, which complicates Cartesian path tracking of the end-effector. Supervised data-driven control is difficult to apply because labeled on-orbit data are scarce. This work therefore benchmarks six reinforcement learning (RL) algorithms (TRPO, TD3, SAC, PPO, PG, and DDPG) for Cartesian path tracking of a free-floating space robot. The benchmark uses a physics-based Simulink/Simscape environment with integrated runtime safety monitoring. Performance is evaluated using a unified set of key performance indicators covering training duration and stability, tracking accuracy, base attitude disturbance, motion smoothness, energy consumption, and collision risk. Among the six agents with default hyperparameters, PPO yields the most favorable trade-off between tracking accuracy, base disturbance minimization, and training stability. The effect of tuning its hyperparameters is then quantified for circular and piecewise-linear Cartesian paths. Compared with the default configuration, the optimized PPO agent increases the mean episodic return from -2.23 to -0.43 . It reduces the mean squared and maximum tracking errors by 67.7% and 44.1%, the base orientation error by 67.0%, and the energy consumption by 16.0%. No collisions or joint-limit violations are observed in the 30 evaluation episodes considered. All results are obtained in simulation under controlled assumptions (ideal sensing, zero latency, rigid-body dynamics, and no external disturbances in the nominal setup) and do not establish operational readiness. The safety component comprises runtime monitors for selected properties and an offline torque-bound check on a simplified model, not a formal proof of closed-loop safety.

INDEX TERMS Free-floating on-orbit servicing, Proximal Policy Optimization (PPO), reinforcement learning, safety monitoring, space robotics, trajectory tracking

I. INTRODUCTION AND BACKGROUND

Free-floating robot manipulators operating in space for satellite inspection, capture, or de-orbiting introduce control challenges not encountered with terrestrial robots [1]. In micro-gravity, the robot's base (typically a spacecraft) is not fixed, and the thrusters are inactive. Manipulator motions therefore change the position and orientation of the base, which in turn modifies the pose of the end-effector. This reaction is a consequence of the conservation of linear and angular momentum [2]. Operating the manipulator under free-floating dynamics avoids the consumption of propellant (e.g., hydrazine, N_2H_4) and the excitation of structural modes by the pulse-width modulation of thruster control at certain frequencies [3]. This can extend the operational lifetime of the spacecraft.

However, a free-floating base complicates manipulator

control. Standard motion-planning approaches developed for fixed-base robots can fail to drive the end-effector to the desired target. Additional constraints, such as communication latency and limited onboard computational resources, further complicate continuous real-time control and base stabilization [1], [4]. Model-based approaches that assume an accurate dynamic model (e.g., reactionless planning or precise attitude control) are sensitive to model uncertainty and unmodeled dynamics. Supervised data-driven approaches are also difficult to apply because labeled training data of sufficient amount and quality are typically unavailable in the space sector. In addition, tracking capability under a free-floating base often has to be demonstrated before the physical system is assembled and deployed [5]–[7].

These challenges motivate the development and simulation



FIGURE 1: Virtual testbeds based on AI-driven digital twins. They allow learning-based control methods for in-space operations to be developed under repeatable and controllable conditions. Illustration created using a picture of the physical setup and with the help of GPT-Image-2.

of AI-enabled digital twins (DTs) [8], as illustrated in Fig. 1. Unlike standard DTs that primarily mirror sensed real-time data from physical counterparts, the DTs considered here combine physics-based simulation with learning components to contextualize system states and evaluate representative operating conditions. These conditions include uncertain dynamics, disturbances, faults, and contact-rich interactions associated with satellite capture. Faster-than-real-time simulation and systematic ablation studies can support controller design and help assess sim-to-real effects. In this role, AI-enabled DTs can provide a training and validation environment for learning-based controllers, in particular when flight data are limited and the constraints are safety-critical.

Reinforcement learning (RL) has been increasingly applied in robotics. An RL agent learns a control policy through trial-and-error interaction with its environment. Recent advances in deep RL have addressed a range of complex continuous-control tasks, although applications to space robots remain comparatively recent [9], [12]–[14], [16].

This work investigates continuous-action deep RL for Cartesian path tracking of the end-effector of a free-floating space manipulator. The contribution differs from prior work in three respects. First, six RL algorithms are compared under the same conditions (see Section VI), complemented by ablation studies (see Section VII-B). The works listed in Table 1 do not provide such a comparison. Second, selected constraints such as collisions and joint limits are monitored at runtime with formal-methods-based tools integrated into the simulation toolchain (see Section V). Third, base attitude minimization during arm motion is treated as an explicit control objective in conjunction with the constraint monitoring above. The induced base disturbance is not addressed in most of the works listed in Table 1. To the authors' knowledge, a unified comparison of RL algorithms for Cartesian path

tracking of free-floating space robots that jointly considers base attitude minimization and formal-methods-based constraint monitoring remains unavailable.

As shown in Table 1, the surveyed prior contributions typically evaluate a single RL algorithm on a specific subtask, which limits conclusions about the relative merits of different learning algorithms for free-floating space robot control. In addition, the surveyed works do not combine three aspects. These are (i) a multi-algorithm benchmark under unified conditions, (ii) explicit minimization of the base attitude disturbance alongside end-effector tracking, and (iii) formal-methods-based runtime safety monitoring in the RL training and evaluation loop. This work addresses these aspects together in a reproducible simulation benchmark with runtime monitoring of selected safety properties, which extends the individual studies listed in Table 1.

The main contributions of this work are summarized as follows:

- *Comparative evaluation of RL agents.* Six continuous-action RL algorithms are implemented and evaluated on the same path-tracking task. The algorithms are Trust Region Policy Optimization (TRPO) [17], Twin Delayed Deep Deterministic Policy Gradient (TD3) [18], Soft Actor-Critic (SAC) [19], Proximal Policy Optimization (PPO) [20], vanilla Policy Gradient (PG) [21], and Deep Deterministic Policy Gradient (DDPG) [43]. The comparison is performed under unified conditions and examines which algorithm offers the most favorable trade-off between tracking accuracy, base disturbance, and training stability.
- *Integration of formal-methods-based safety monitoring.* Formal-methods tooling is integrated into the learning loop [2], [23]–[25]. A collision monitor, a joint-limit monitor, and a momentum-conservation check run at runtime, and the joint-torque bound is additionally checked offline with Simulink Design Verifier on a simplified abstraction of the policy. This combination monitors selected safety properties (collision events, joint limits, and bounded actuator commands) during training and testing. It does not constitute a formal proof of closed-loop stability, task completion, or collision avoidance for the full nonlinear multibody model.
- *Simulation environment for multi-criteria evaluation in on-orbit servicing.* A MATLAB/Simulink simulation framework is developed for a free-floating space robot. It uses the Simscape physics engine and a multibody model derived from a Unified Robot Description Format (URDF) file. The robot consists of a cuboid base and a 4-degree-of-freedom (4-DOF) revolute arm, modeled with consistent kinematic, inertial, and sensor/actuator parameters. The associated RL environment provides observations and rewards through a composite reward function that penalizes end-effector tracking error, base motion, large joint velocities, and excessive control effort. Together with the reset conditions, this reward shaping allows the agent to learn the task while penal-

TABLE 1: Recent advances in deep RL-based motion management for free-floating space robots. Abbreviations: RL Alg.=reinforcement learning algorithm; DOF=degrees of freedom; Obs./Act. Dim.=observation-/action-space dimension; vel.=joint velocities; rates=joint rate commands; pos./att.=position/attitude; pos./ori.=position/orientation; EE=end-effector; Formal Meth.=formal-methods-based safety components; Base Dist.Min.=base disturbance minimization; d_{safe} =minimum admissible distance; τ_{max} =torque bound; K_1 – K_9 =evaluation KPIs (see Section IV); N/R=not reported; [†]runtime monitors for selected properties and an offline torque-bound check on a simplified model, not a formal proof of closed-loop safety.

Contribution	RL Alg.	Task Objective	DOF	Simulator	Obs./Act. Dim.	Safety Limits	Eval. Metrics	Formal Meth./ Base Dist. Min.
[9], 2023	SAC	Obstacle avoidance	7	OpenAI Gym	14/7D vel.	Reward-based	Success rate	No/No
[10], 2024	DDPG	Collision avoidance	6	Custom	N/R/6D vel.	Reward-based	Success rate	No/No
[11], 2024	PPO	Joint failure adaptation	7	MATLAB Simscape + SPART	32/7D rates	Pos./att. threshold	Per-joint success rate	No/No
[12], 2024	DDPG	Path planning	6	Custom	32/6D vel.	Reward-based	Convergence, task completion	No/No
[13], 2025	PPO	Trajectory planning	6	PyBullet	N/R/6D	Joint limits	Success rate, pos./ori. error	No/No
[14], 2025	LSAC	Trajectory post-processing	N/R	Custom + air-bearing	N/R/N/R	Uncertainty bounds	Tracking error	No/No
[15], 2025	DDPG	Trajectory planning	N/R	Custom	N/R/N/R	Base dist. (soft)	Reward, base dist. red.	No/Yes
Ours	PG, PPO, TRPO, DDPG, TD3, SAC	Cartesian EE tracking	4	MATLAB/Simulink + Simscape	23/4D τ	$d_{\text{safe}}, \tau_{\text{max}},$ joint limits	K_1–K_9	Partial[†]/Yes

izing base disturbance and violations of the monitored constraints [22], [26]–[29].

- *Optimization of the PPO agent.* Based on the comparative results, PPO is selected for further tuning. Hyperparameters such as the learning rates, the rollout horizon, the mini-batch size, and the entropy weight are adjusted to improve training stability and policy performance. The hidden layers of the default networks are kept. Relative to the default configuration, the optimized PPO agent reduces the tracking error and produces smoother control inputs on the tested KPIs. No violations of the monitored safety properties are observed in the 30 evaluation episodes.

Within the considered simulation setup, the combination of multibody simulation, deep RL, and runtime safety monitoring yields control policies without observed violations of the monitored constraints in the evaluation episodes. The optimized policy also reduces the tracking error and the base disturbance relative to the default configuration. The remainder of this work is organized as follows. Section III describes the system model and the simulation environment. Section IV introduces the reinforcement learning framework. Section V presents the safety monitors and formal-methods components. Section VI reports the comparative evaluation of the six RL algorithms. Section VII describes the optimization of the PPO agent and its evaluation on circular and piecewise-linear references. Section VIII discusses the results and concludes the work.

II. RELATED WORK

A. CLASSICAL CONTROL TECHNIQUES FOR SPACE ROBOTS

Model-based approaches have historically dominated space robot control. Computed torque control and resolved-motion rate methods use accurate dynamic models to achieve precise end-effector positioning [1]. Reactionless path planning explicitly minimizes base disturbance. It exploits the robot's redundancy and a dynamics-related null-space to design the joint velocities. Attitude regulation methods instead use reaction wheels or thrusters to counteract the momentum transfer [2], [7]. These approaches offer strong performance guarantees and interpretability when the system model is accurate. However, they are sensitive to model uncertainties and unmodeled dynamics and require a detailed model identification [37]. They are therefore difficult to apply under the data-scarce and hazardous conditions of on-orbit operations.

B. REINFORCEMENT LEARNING-BASED CONTROL TECHNIQUES

RL has been applied successfully to various robotic manipulation tasks [22], [30]. Policy gradient methods (PG, PPO, TRPO) and actor-critic algorithms (DDPG, TD3, SAC) have been widely applied to continuous control problems [17]–[21]. Applied to free-floating space robots, RL-based methods have been investigated for path planning [12], [16], obstacle avoidance [9], trajectory tracking [13], and failure management [11]. RL does not require an explicit dynamic model of the system, which is relevant when model parameters are uncertain or unavailable. However, current RL-based approaches share recurring limitations. Most studies evaluate only a single algorithm, address a single objective, omit base attitude minimization, and do not provide systematic benchmarking under comparable conditions.

C. HYBRID SAFETY-AWARE CONTROL SYSTEMS

Safe RL integrates safety constraints through formal verification, barrier functions, and constrained optimization [24], [25], [31], [32]. Shielding approaches [25] intercept unsafe actions at runtime and substitute corrective commands, while control barrier functions (CBFs) [32] encode safety as a continuously enforced constraint on the policy's action set. These hybrid methods combine the adaptability of RL with formally specified runtime constraints. They help to enforce properties such as joint-limit compliance and collision avoidance. However, they have rarely been applied to free-floating space robots. Existing works seldom integrate formal-methods-based safety monitoring into the RL training loop or combine it with base attitude minimization objectives.

D. NEED FOR BENCHMARK STUDIES

Comparative evaluations of RL algorithms for space robot control are scarce. Table 1 surveys recent deep RL contributions to free-floating space robot motion management. As shown, all surveyed works address only individual goals with a single algorithm. None simultaneously combines multi-algorithm and multi-criteria benchmarking, base disturbance minimization, and formal-methods-based safety monitoring. This absence of unified evaluation makes it difficult to draw conclusions about relative algorithm performance and to guide algorithm selection for future missions.

E. RESEARCH GAP

The survey above points to three gaps in the existing literature. First, none of the surveyed works compares multiple RL algorithms under identical conditions for Cartesian end-effector tracking of free-floating space robots. Second, base attitude minimization, a safety-critical requirement in on-orbit operations, is omitted in most RL-based contributions (see Table 1). Third, the integration of formal-methods-based safety monitoring into the RL training and evaluation loop has not been reported for this class of systems in the surveyed works. This work addresses the three gaps together within a controlled simulation benchmark. It benchmarks six RL algorithms under unified conditions and treats base attitude minimization as an explicit control objective. Runtime monitoring of selected safety properties is integrated into the RL training and evaluation loop. The actuator-torque bound is additionally checked offline on a simplified verification model.

III. SYSTEM MODELING AND SIMULATION ENVIRONMENT

A. ADVANTAGES OF AI-ENDOWED DIGITAL TWIN TECHNOLOGY IN ON-ORBIT SERVICING

AI-driven DTs have been applied in on-orbit servicing, for example for the automated lifecycle management of satellites and for the orchestration of agents that carry out assembly, integration, and testing tasks [33], [34]. DTs combine contextualized real-time sensor data with AI methods that account

TABLE 2: Parameters of the simulated free-floating space robot.

Parameter	Value
Base, cube with 1 m edge length, mass	30 kg
Base, principal moments of inertia	6 kg m ²
Links (4), length and mass	1 m, 2 kg
Links, principal moments of inertia	0.2 kg m ²
Viscous joint damping	1.5 N m s/rad
Joint limits q_1 and q_2 to q_4	$\pm 85^\circ$ and $\pm 170^\circ$
Joint-limit stiffness and damping	3000 N m/rad, 40 N m s/rad
Torque limit $\tau_{\max}$	2 N m
Gravity	$[0 \ 0 \ 0]^T$

for uncertainty. This allows simulation-based analysis under various operational conditions at reduced cost. Verification and validation can also benefit, because changes in system design or operational scenarios can be described in scenario spaces and linked to DTs [35].

In this work, the DTs of the target satellite and the chaser space robot are based on an information model. It provides a structured and interoperable description of both systems, including their kinematics, inertial properties, joint configurations, and physical constraints. The AI capability is provided by the deep RL agents, which are trained and evaluated through interaction with the corresponding DTs and with additional DTs of representative in-space operation scenarios. This distinguishes the approach from classical simulation with pre-specified controllers. The robotics-related DT hosts a learning-based agent that derives its control strategy from experience. The DTs are therefore used as training and validation environments for learned control strategies. During training, the initial joint configuration of each episode is sampled from a bounded uniform distribution. The agent therefore learns from a range of admissible initial conditions instead of a single fixed start.

B. ROBOT MODEL

The virtual testbed simulates a free-floating space manipulator derived from the Spacecraft Robotics Toolkit (SPART) example [36]. The robot consists of a cuboid base (serving as the spacecraft) and a 4-DOF serial manipulator (see Fig. 2). The arm has four revolute joints and four links. Its kinematic structure and inertial properties are specified in a URDF file (`SpaceRobot.urdf`) imported into Simulink's multibody environment [26]–[28]. The geometry and the inertial parameters of the base and the links are listed in Table 2. The same values are used in the URDF file and in the Simscape model.

The resulting multibody model captures the dynamic coupling between the floating base and the arm [2].

C. DYNAMIC SIMULATION IN MATLAB/SIMULINK

The simulation is implemented in Simulink using the Simscape Multibody physics engine. Gravity is set to $[0 \ 0 \ 0]^T$ to emulate the microgravity orbital environment. The base is connected to the inertial world frame through an unactuated 6-DOF joint [38], allowing free translation and rotation. No external forces or torques act on the system, so the base

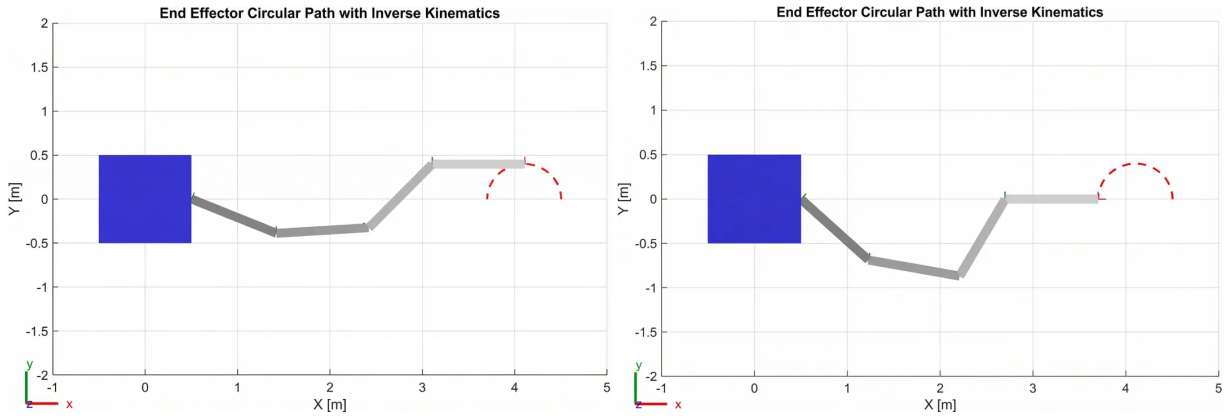
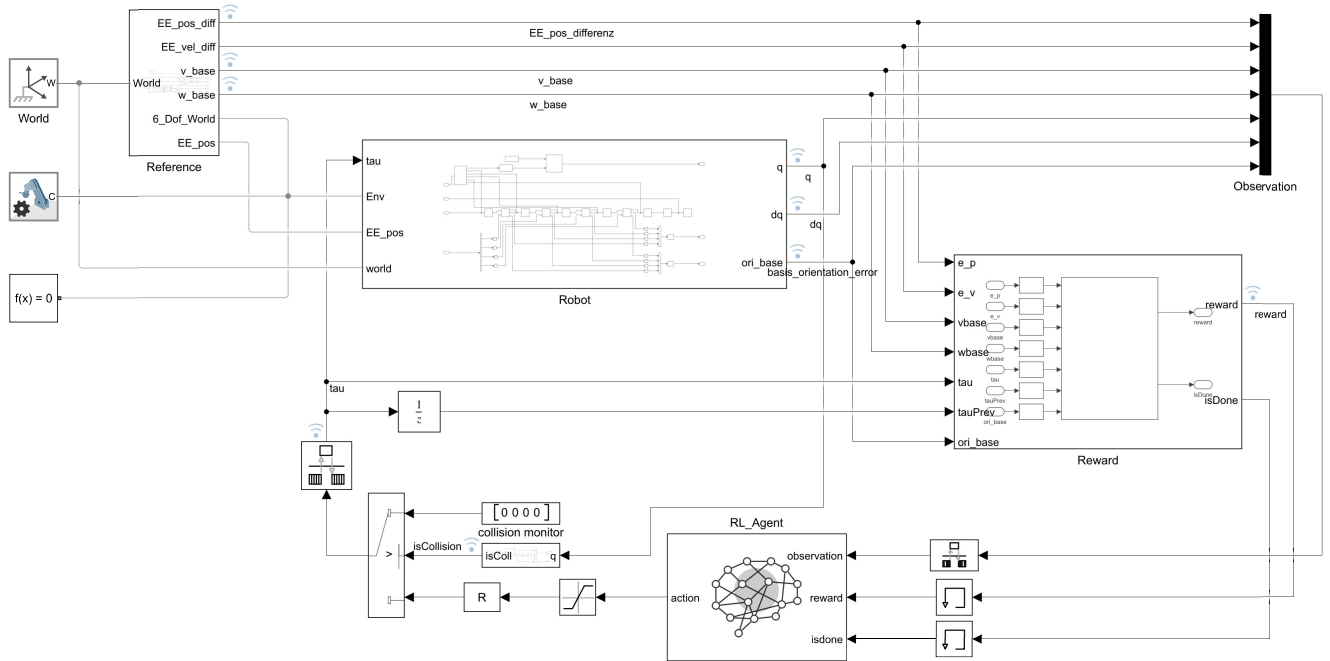


FIGURE 2: Tracking the desired end-effector circular path using inverse kinematics.

FIGURE 3: Overview of the Simulink model `SpaceRobot.slx`. The closed-loop model of the free-floating space robot integrates the robot dynamics, reference generation, observation assembly, reward computation, and a PPO-based RL agent that outputs joint torques. The reward and termination logic evaluate end-effector tracking, base motion, and control effort.

moves only in response to the internal torques generated by the manipulator joints. Under this configuration, the total linear and angular momentum are conserved, and arm motions induce a reactive motion of the base [2], [7], [29], [32], [38].

The simulation is based on idealizing assumptions that are common in RL-based space robotics research. They are stated here explicitly to support reproducibility and a fair comparison. First, ideal sensing is assumed. All state variables (joint angles, joint velocities, base orientation quaternion, and base angular velocity) are read directly from Simulink signal buses without sensor noise, quantization errors, or measurement bias. Second, zero communication latency is assumed. The RL agent receives observations and issues torque commands within the same discrete time step,

with no transmission delay between sensing and actuation. Third, the robot is modeled as a rigid multibody system. Flexible link dynamics, joint compliance, and thermal drift are not included. Fourth, no external disturbances act on the spacecraft. Gravity gradients, atmospheric drag, solar radiation pressure, and residual magnetic torques are all set to zero, consistent with the free-floating assumption in low-Earth orbit. These assumptions give a controlled benchmark environment. Performance differences can therefore be attributed mainly to the algorithms rather than to environmental variability. Consequently, the reported results characterize relative algorithm performance in a simulation benchmark under controlled assumptions. They do not demonstrate robustness to real sensing, communication latency, structural

TABLE 3: Comparison of simulation platforms relevant to free-floating space robotics with formal safety monitors.

Criterion	MATLAB/ Simulink	Gazebo	MuJoCo
Physics engine	Simscape Multibody	ODE, Bullet, DART	MuJoCo (na- tive)
Free-floating multibody fidelity	High	Medium	High
Formal verification integration	Native (Stateflow, Design Verifier)	None	None
RL framework	RL Toolbox	gym-gazebo, ROS bridges	Gymnasium, dm_control
Sensor and ROS inte- gration	Limited (via add-ons)	Extensive	Limited
Code generation / HIL path	Mature (Embedded Coder)	Via ROS 2	Limited
Licensing	Commercial	Open source	Open source

flexibility, or orbital disturbances, and they do not constitute an assessment of operational readiness for on-orbit deployment. A limited stress test under non-ideal conditions is reported in Section VII-D. The impact of the assumptions on real-world deployment is discussed in Section VIII.

MATLAB/Simulink was chosen as the simulation environment after considering two alternatives that are common in robotics and RL research, Gazebo [50] and MuJoCo [49]. MuJoCo offers an efficient contact and multibody solver and is widely used for RL benchmarks in continuous control [51]. However, it has no native interface to Simulink, Stateflow, and the model-based verification tools that this work uses for the safety components. Gazebo provides mature Robot Operating System (ROS) integration and plug-in-based sensor models and is well suited for hardware-in-the-loop (HIL) development [52]. Its general-purpose multibody back-ends are, however, less convenient for free-floating spacecraft scenarios, and it offers no native path to formal verification tools. Simscape Multibody provides a multibody solver for the free-floating base-manipulator coupling. It is directly connected to Stateflow and Simulink Design Verifier, which this work uses for the safety components. Together with the Reinforcement Learning Toolbox and a deterministic fixed-step solver, this gives a single reproducible toolchain from the physical model to the learned policy and the safety checks. Table 3 summarizes the trade-off. This choice gives up the higher simulation speed of MuJoCo and the richer sensor and ROS tooling of Gazebo in exchange for a close coupling of the dynamics, the RL agent, and the formal-methods tools.

The Simulink model is composed of several subsystems, as illustrated in Fig. 3. The Robot Plant subsystem computes the multibody dynamics of the free-floating robot at each time step (see Fig. 4) by numerically integrating the equations of motion for the applied joint torques [2]. The Controller subsystem executes the learned RL policy. The commanded joint torques pass through a saturation block and a switch before reaching the plant. When the collision monitor reports

a contact, the switch overrides the torques to zero [24], [25]. The Reference Trajectory Generator provides the desired end-effector trajectory and base attitude (see Fig. 5), outputting the current target position and velocity at each time step. A set of sensors measures the robot state (joint angles, joint velocities, base orientation, and base angular velocity) and assembles the observation signal for the RL agent (detailed in Section IV). Finally, two safety monitors check the operational constraints at every agent step. The collision monitor computes the minimum distance between non-adjacent robot bodies and detects contacts. The joint-limit monitor compares every joint angle with its admissible range. A detected contact or a joint-limit violation terminates the episode [24], [25], [39]. Details are given in Section V.

To couple the continuous dynamics with the discrete-time RL agent, a consistent integration scheme is used. The model is simulated with a fixed-step solver (ode4) and a step size of $T_s = 0.01$ s instead of the default variable-step solver. The agent acts every $\Delta t = 0.1$ s. Rate transition blocks connect the two rates and hold each torque command constant over the ten solver steps of an agent interval [40], [41]. Simulink unit consistency checks are enabled to detect modeling errors such as mismatched units early in development.

D. REFERENCE TRAJECTORY AND DESIRED TASK

The task is to follow a semicircular end-effector trajectory while minimizing base disturbance. The reference starts at the end-effector position of the stretched arm at (4.5, 0, 0) m and follows a semicircle with radius $r = 0.5$ m around (4.0, 0, 0) m within $T = 8.5$ s (see Fig. 2). Before training, the trajectory was checked with inverse kinematics for geometric feasibility, that is, for compliance with the joint limits and the absence of singularities. The inverse-kinematics solution gave a consistent set of joint trajectories that realize the desired end-effector motion. The path is therefore kinematically feasible. A failure of the RL agent can thus be attributed to learning and not to a kinematically infeasible reference. During simulation, the reference subsystem outputs the desired end-effector position and velocity as time series $EE_{\text{ref}}(t)$ and $EE_{v,\text{ref}}(t)$ at each time step. The instantaneous position and velocity tracking errors are derived from these signals. They enter both the observation vector and the reward computation that penalizes large errors.

IV. REINFORCEMENT LEARNING FRAMEWORK

State (Observation) Space. At each time step the RL agent receives an observation that summarizes the state variables relevant for control [22]. The state vector consists of the following components:

- *End-effector tracking error.* The Cartesian position error $e_p \in \mathbb{R}^3$ between the current and reference end-effector positions, together with the velocity error $e_v \in \mathbb{R}^3$. When the end-effector is on the reference trajectory, $e_p = 0$. The pair (e_p, e_v) indicates both the spatial deviation from the reference and the velocity mismatch.

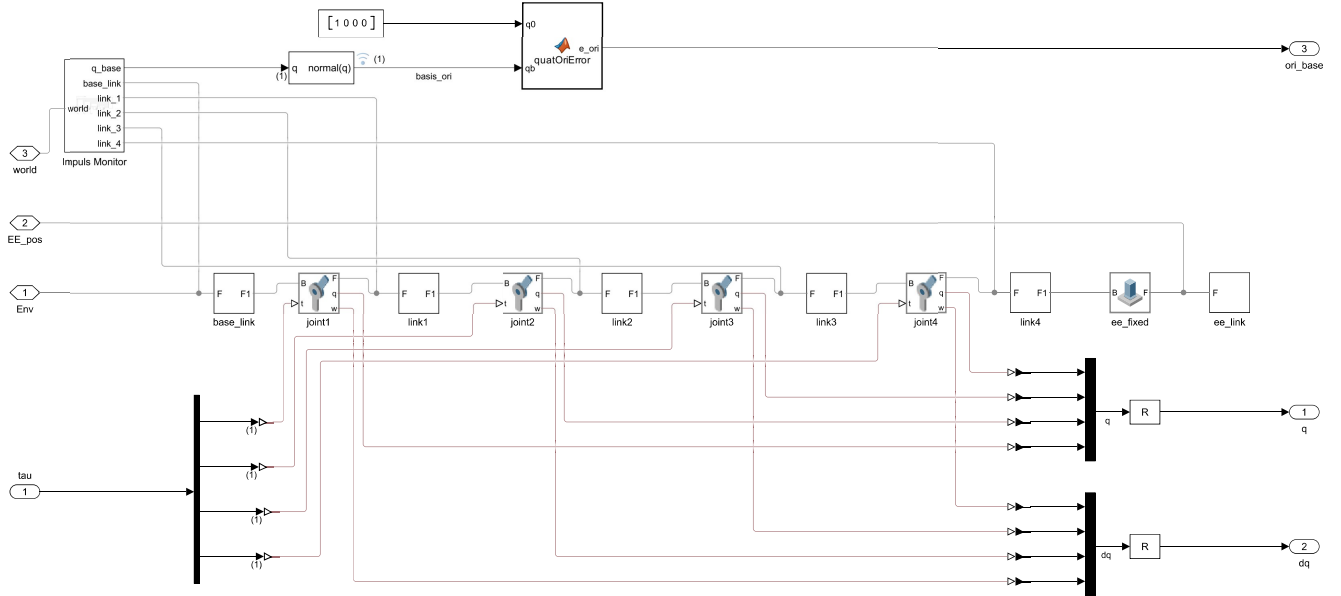


FIGURE 4: Robot subsystem of the Simulink model *SpaceRobot.slx*. The Simscape Multibody model of the free-floating space robot consists of a floating base and a serial 4-DOF manipulator. Joint torque commands are applied to the revolute joints, and the joint positions and velocities are measured and fed back as state variables. The base orientation is represented as a unit quaternion and compared with a reference to compute the base attitude error.

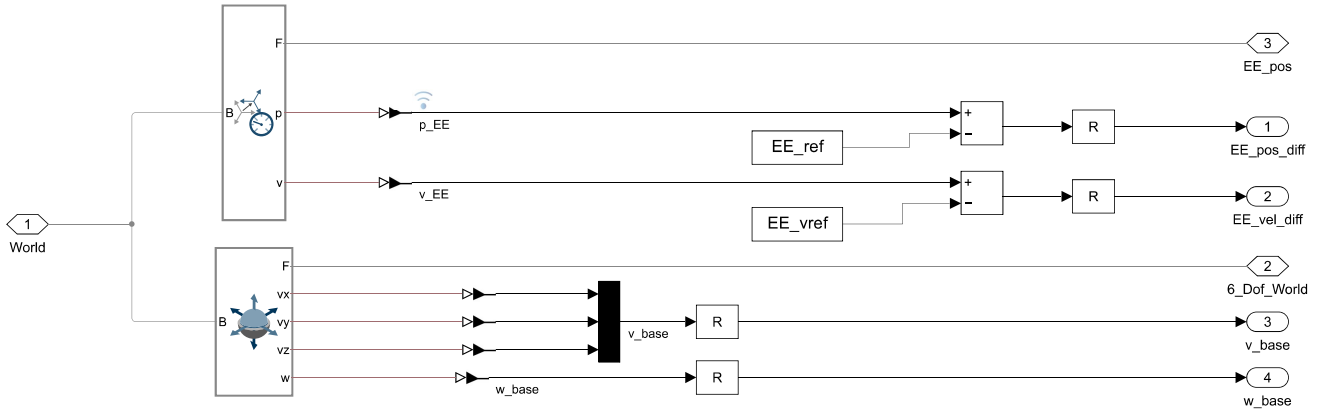


FIGURE 5: Reference subsystem of the Simulink model *SpaceRobot.slx*. The measured end-effector position and velocity are compared with the reference signals. The resulting error vectors are used for the observation and the reward. The base linear and angular velocities are also extracted to capture the base motion induced by the manipulator.

- *Arm joint states.* The joint angles $q \in \mathbb{R}^4$ and joint angular velocities $\dot{q} \in \mathbb{R}^4$. These describe the configuration and motion state of the manipulator. Joint angles are provided in radians together with the corresponding mechanical limits.
- *Base motion.* The orientation, angular velocity, and linear velocity of the free-floating base. The base attitude error is computed from the measured base quaternion q_b and the reference quaternion q_0 as the relative quaternion $q_{err} = q_0^* \otimes q_b$. Its sign is chosen so that the scalar part is non-negative. The relative quaternion is then mapped to the axis-angle rotation vector $e_{ori} = \theta \mathbf{u} \in \mathbb{R}^3$ with $\theta = 2 \operatorname{atan2}(\|q_{err,v}\|, q_{err,w})$

and $\mathbf{u} = q_{err,v} / \|q_{err,v}\|$. This 3-dimensional minimal representation removes the redundancy of the unit-quaternion parameterization and is well-defined for $q_{err} \neq \pm[1, 0, 0, 0]^T$. The base should keep its initial attitude, so the agent has to keep e_{ori} close to zero. The base angular velocity $\omega_{base} \in \mathbb{R}^3$ and the base linear velocity $v_{base} \in \mathbb{R}^3$ are also included to capture the residual rotation and the momentum-coupled translation of the base.

The overall observation vector has dimension $3(e_p) + 3(e_v) + 4(q) + 4(\dot{q}) + 3(\text{ori}_{base}) + 3(\omega_{base}) + 3(v_{base}) = 23$. This state representation contains the variables required by the agent to assess tracking deviation, arm configuration, and

base motion.

Action Space. The action is a vector of continuous joint torque commands. At each decision step, the agent outputs $a = [\tau_1, \tau_2, \tau_3, \tau_4]^T$, with one component per revolute joint of the arm [22], [30]. The actions are real-valued and bounded by the actuator capabilities. Symmetric torque limits $|\tau_i| \leq \tau_{\max}$ for $i = 1, \dots, 4$ are enforced through saturation blocks that clip the commanded torque to the interval $[-\tau_{\max}, +\tau_{\max}]$. The value of $\tau_{\max}$ is selected according to the robot design ($\tau_{\max} = 2 \text{ N m}$ in this work), provided it is large enough to follow the reference trajectory. The saturation enforces both the actuator limits and an upper bound on the applied torque for safety reasons. The action space is therefore $\mathbb{R}^4$, which is compatible with policy gradient and actor-critic methods.

The policy is a mapping from the 23-dimensional state space to the 4-dimensional action space, $\pi : s \mapsto a$, parameterized by a neural network whose parameters are adjusted during training to maximize the expected cumulative reward. The agent issues a new torque command at a fixed interval $\Delta t = 0.1 \text{ s}$. With an episode duration of 8.5 s , this gives $N = 85$ decisions per episode.

Reward Function Design. The reward signal guides the agent toward accurate trajectory tracking. At the same time, it penalizes base disturbance and favors smooth and bounded control actions. At each discrete time step t , the reward r_t is computed as the sum of a progress term and a proximity bonus, minus a weighted cost function:

$$r_t = \frac{1}{C} (r_{\text{prog}} + r_{\text{bonus}} - c_t), \quad (1)$$

where C is a normalization constant.

To encourage monotonic convergence toward the reference trajectory, a progress reward is defined based on the reduction of the end-effector position error:

$$r_{\text{prog}} = \begin{cases} k_p (d_{t-1} - d_t), & d_{t-1} > d_t, \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

where $d_t = \|e_p(t)\|$ is the Euclidean end-effector position error and k_p is a scaling factor. The previous distance d_{t-1} is stored persistently across time steps.

A small shaping bonus is added when the end-effector is close to the reference point:

$$r_{\text{bonus}} = k_b \exp\left(-\left(\frac{d_t}{\sigma}\right)^2\right), \quad (3)$$

which is intended to improve the local convergence near the desired trajectory.

The instantaneous cost c_t penalizes tracking errors, base motion, and control effort:

$$c_t = W_p \|e_p\|^2 + W_v \|e_v\|^2 + W_{\text{ori}} \|e_{\text{ori}}\|^2 + W_{\omega_b} \|\omega_{\text{base}}\|^2 + W_{v_b} \|v_{\text{base}}\|^2 + W_u \|\tau\|^2 + W_d \|\tau - \tau_{t-1}\|^2, \quad (4)$$

where e_p and e_v denote the end-effector position and velocity errors, e_{ori} the base orientation error, v_{base} and ω_{base} the

linear and angular base velocities, and τ the joint torque vector. The final term penalizes changes in control input to promote smooth actuation.

The weights used in all experiments are summarized in Table 4. The physical intuition behind the weight choices is as follows. The end-effector position error weight $W_p = 150$ is large because trajectory tracking is the primary task objective. A high weight makes the position error the dominant term of the cost at typical error magnitudes. The velocity error weight $W_v = 25$ is lower than W_p to avoid over-penalizing transient dynamics during fast arm motion, while still discouraging persistent velocity offsets. The base orientation weight $W_{\text{ori}} = 200$ is the largest weight in the cost function, because base attitude disturbance is safety-critical in on-orbit operations. An uncontrolled rotation of the spacecraft base can endanger the mission regardless of the tracking performance. The base angular velocity weight $W_{\omega_b} = 8$ penalizes ongoing base rotation, similar to a damping term. The linear velocity weight $W_{v_b} = 2$ is kept small. With zero initial linear momentum, base translation is coupled to the manipulator motion and is expected to remain small for the considered task. It cannot be avoided completely, because the center of mass of the whole system stays fixed. The torque magnitude weight $W_u = 0.02$ and the torque-rate weight $W_d = 0.06$ are small so that the agent keeps enough freedom to explore different control actions. The torque-rate penalty is slightly larger to discourage high-frequency chattering in the control signal. The progress scaling factor $k_p = 10$ is chosen so that the progress reward has a magnitude comparable to the cost terms at typical error levels and is not dominated by them. The proximity bonus parameters $k_b = 0.05$ and $\sigma = 0.02$ are small by design. They provide a weak local guidance signal near the trajectory without creating a strong local attractor that could trap the policy. Finally, the normalization constant $C = 500$ scales the total reward to a range of approximately $[-1, 0]$ under typical operating conditions, which is intended to stabilize the value-function learning of all tested algorithms.

At each step, all inputs to the reward function are checked for numerical validity. An episode is terminated early if a NaN or Inf value is detected, if the end-effector error exceeds 10 m , if the collision monitor reports a contact, or if a joint leaves its admissible range. In that case, the agent receives the lower end of the one-step reward range, -1 , for the terminal step and for every remaining step of the episode. The terminal penalty is therefore

$$r_{\text{fail}} = -(N - k + 1), \quad (5)$$

where k is the step at which the violation occurs and $N = 85$ is the number of steps per episode. A constant penalty is not sufficient in this setting. The step rewards are negative in nearly all states, so a single penalty of -1 would make an early termination more attractive than continuing with a poor policy. In a pilot run with a constant penalty of -1 , none of the last 100 training episodes of the default and the optimized PPO agent reached the end of the horizon. With

the penalty in (5), all of them did. All reward components were implemented in a MATLAB Function block within the Simulink environment.

Training Procedure. An episodic training scheme is used. Each episode covers the semicircular reference once and lasts $T = 8.5$ s, which corresponds to $N = 85$ agent steps. At the start of each episode, a custom reset function sets the base to its home orientation at rest. The initial joint angles are drawn uniformly within $\pm 1^\circ$ around the nominal stretched configuration $q = 0$, in which the end-effector is located at the start of the reference. These initial states lie close to a configuration that is free of collisions and far from the joint limits. All state variables, integrator states, and logging signals are reset as well [22].

Within an episode, the agent receives an observation at each step and outputs a torque action. The simulation runs until the end of the horizon or until one of the terminal conditions of the reward function occurs. In that case the penalty in (5) is applied and the episode ends at that step.

The training loop is provided by MATLAB's Reinforcement Learning Toolbox. After each episode or after a fixed number of steps, the collected states, actions, and rewards are used to update the policy according to the chosen RL algorithm [42]. Where applicable, the value function is updated as well. The algorithm-specific update rules of PPO, DDPG, and the remaining methods are handled by their respective toolbox implementations. This work supplies the experience and the reward signal [22]. All agents are trained for the same number of episodes, $N_{\text{train}} = 1000$. This budget is intended to let most algorithms converge or reach a performance plateau. During training, the cumulative reward and the length of every episode are recorded. A successful training run is characterized by an episode return that increases toward zero, since the step rewards are negative costs. An episode that ends before N steps was terminated by a monitor, so the episode length also shows how often safety violations occur during training.

To make the runs reproducible, MATLAB's Mersenne Twister random number generator is initialized with a seed s before each training run (`rng(s, 'twister')`). The seed determines the initial network weights, the exploration noise, the sampling of mini-batches, and the initial joint angles of every episode. Two runs with the same seed are identical.

TABLE 4: Reward function weights used in all experiments.

Term	Symbol	Value
End-effector position error	W_p	150
End-effector velocity error	W_v	25
Base orientation error	W_{ori}	200
Base angular velocity	W_{ω_b}	8
Base linear velocity	W_{v_b}	2
Joint torque magnitude	W_u	0.02
Torque rate (smoothness)	W_d	0.06
Progress scaling factor	k_p	10
Proximity bonus scaling	k_b	0.05
Proximity width	σ	0.02
Reward normalization constant	C	500

Each configuration was trained with ten seeds ($s = 0, \dots, 9$). This results in 70 training runs for the six default agents and the optimized PPO agent. Ten seeds are used so that the seed-level tests keep some resolution after the correction for multiple comparisons. Every run comprises exactly $N_{\text{train}} = 1000$ episodes without early stopping, and the policy after the last episode is evaluated. No checkpoint is selected, and all ten seeds are reported.

Evaluation and Metrics. After training, each policy is evaluated deterministically. For the stochastic policies, the mean action is used. Every policy is evaluated on $N_{\text{eval}} = 30$ episodes whose initial joint angles are drawn uniformly within $\pm 1^\circ$ from a fixed random stream, and on one additional episode from the nominal start pose. All agents use the same initial states, so the evaluation episodes are paired across agents. The conclusions therefore remain conditional on these initial states and the tested reference trajectories. Over the evaluation episodes, a set of Key Performance Indicators (KPIs) is computed to compare the agents quantitatively. For episodes that terminate early due to a safety violation, each KPI is computed over the executed portion of the episode up to the termination step. The KPIs are first computed for each seed over the 30 random episodes. The seed dependence and the performance reached are then reported separately. A training run counts as successful when none of its 31 evaluation episodes is terminated by a monitor. The first table reports the number of successful runs per configuration and the share of evaluation episodes that a monitor terminated, taken over all runs. The KPI tables report the mean and the standard deviation over the successful runs only, together with their number. These KPI values are therefore conditional on a successful run. Algorithms are compared on the seed level with two-sided Mann–Whitney U tests over the successful runs and a Holm correction for multiple comparisons. A test is reported only when both configurations have at least four successful runs. The shares of successful runs are compared with Fisher's exact test, also with a Holm correction.

The nine KPIs are organized into three groups to aid interpretation. The tracking accuracy group (K_2 and K_3) measures how closely the end-effector follows the reference path. The base stability and safety group covers spacecraft attitude drift (K_4) and constraint violations (K_5 and K_6). The efficiency group covers control smoothness (K_7), residual base rotation (K_8), and energy consumption (K_9). K_1 is an overall scalar summary of the agent's performance across all objectives, since it is the cumulative reward that jointly penalizes all undesired behaviors.

- K_1 (Mean Episodic Return). Mean cumulative reward per episode. Higher is better and reflects overall task success.

$$R_i = \sum_{k=1}^{T_i} r_i[k], \quad K_1 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} R_i.$$

- K_2 (Mean Squared EE Position Error). Average

squared tracking error per episode. Lower is better.

$$K_2 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{p,i}[k]\|_2^2.$$

- **K_3** (Maximum EE Position Error). Worst-case tracking deviation. Lower is better.

$$K_3 = \max_i \max_k \|e_{p,i}[k]\|_2.$$

- **K_4** (Mean Base Orientation Error). Average attitude drift of the spacecraft base, measured as the magnitude of the axis-angle rotation-vector error e_{ori} (in radians). Lower is better.

$$K_4 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|e_{\text{ori},i}[k]\|_2.$$

- **K_5** (Collision Rate). Fraction of episodes with at least one collision. Zero is ideal.

$$C_i = \mathbb{I}(\exists k : c_i[k] > 0.5), \quad K_5 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} C_i.$$

- **K_6** (Joint Limit Violation Rate). Fraction of time steps across episodes where any joint exceeds its mechanical limit. Zero is ideal.

$$K_6 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i n_J} \sum_{k=1}^{T_i} \sum_{j=1}^{n_J} \mathbb{I}(q_{i,j}[k] \notin [q_j^{\min}, q_j^{\max}]).$$

- **K_7** (Control Smoothness (Jerk)). Second difference of joint torques. Lower values indicate smoother, less chattering actuation.

$$\Delta^2 \tau_i[k] = \tau_i[k+2] - 2\tau_i[k+1] + \tau_i[k],$$

$$J_i = \frac{1}{T_i - 2} \sum_{k=1}^{T_i-2} \|\Delta^2 \tau_i[k]\|_2,$$

$$K_7 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} J_i.$$

- **K_8** (Mean Base Angular Velocity). Residual spacecraft rotation rate during the task (rad/s). Lower is better, ideally zero.

$$K_8 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \frac{1}{T_i} \sum_{k=1}^{T_i} \|\omega_{b,i}[k]\|_2.$$

- **K_9** (Energy Consumption). Average absolute mechanical power integrated over each episode. Lower is better when not at the expense of tracking quality.

$$E_i = \int \sum_{j=1}^{n_J} |\tau_{i,j}(t) \dot{q}_{i,j}(t)| dt, \quad K_9 = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} E_i.$$

Two KPIs need a justification in the context of space missions. The control smoothness K_7 captures high-frequency torque oscillations (chattering). In space, such oscillations can accelerate actuator wear and excite structural vibrations.

These vibrations degrade the pointing accuracy of sensitive instruments such as cameras or star trackers. Rapid back-and-forth motion also dissipates energy. Minimizing K_7 therefore supports hardware lifetime and payload pointing requirements. The energy consumption K_9 reflects the mechanical work performed by the joints, approximated as $\int \sum_j |\tau_j \dot{q}_j| dt$. Onboard power is limited by solar panel area and battery capacity. Energy spent on control actuation reduces the budget available for communication, thermal management, and payload operation. Reducing K_9 without sacrificing tracking accuracy therefore has operational value for mission planning.

In addition to the KPIs above, two training-related metrics are reported. T_1 is the standard deviation of the episode reward over the last 100 training episodes, an indicator of training stability. T_2 is the wall-clock training time required for 1000 episodes, an indicator of the computational load.

Applying these metrics to all six agents yields a common basis for comparison. The comparative results are presented and discussed in Section VI.

V. SAFETY MONITORING AND CONSTRAINT ENFORCEMENT

Stability and safety are central requirements in space robotics. A controller for autonomous on-orbit servicing must satisfy the physical constraints of the system and avoid hazardous states, in particular when the control policy is learned from data and must therefore handle uncertainty. In this work, runtime safety monitors and constraint-enforcement mechanisms are integrated directly into the RL training and execution environment. They have two purposes. They terminate episodes that reach unsafe states during exploration, and they shape the agent's behavior toward the monitored constraints through penalties [22], [24]. These mechanisms address a defined set of properties and do not constitute a formal safety certification of the learned controller (see the limitations at the end of this section).

Momentum Conservation Monitor. In a free-floating system without external forces, both the total linear and the total angular momentum of the robot (base and arm) are conserved. With zero initial momentum, the total linear momentum remains close to zero throughout the simulation [2]. A momentum observer computes the total linear momentum p_{tot} and checks for its conservation (see Fig. 6). Virtual sensors attached to the base and to each link provide the body velocities, from which $p_{\text{tot}} = \sum_i m_i v_i$ is computed. In an ideal simulation, p_{tot} remains close to zero throughout an episode. Small numerical deviations on the order of 10^{-6} N s may arise from time-integration errors. A persistent, systematic drift of p_{tot} would indicate an unintended external force or a modeling error, such as an unmodeled constraint. In the reported experiments, p_{tot} remained within 10^{-6} N s of zero throughout each episode with the learned policy (see Fig. 7). This indicates that the simulation conserves momentum within the numerical tolerance. The momentum observer is a diagnostic tool. It is evaluated offline and does

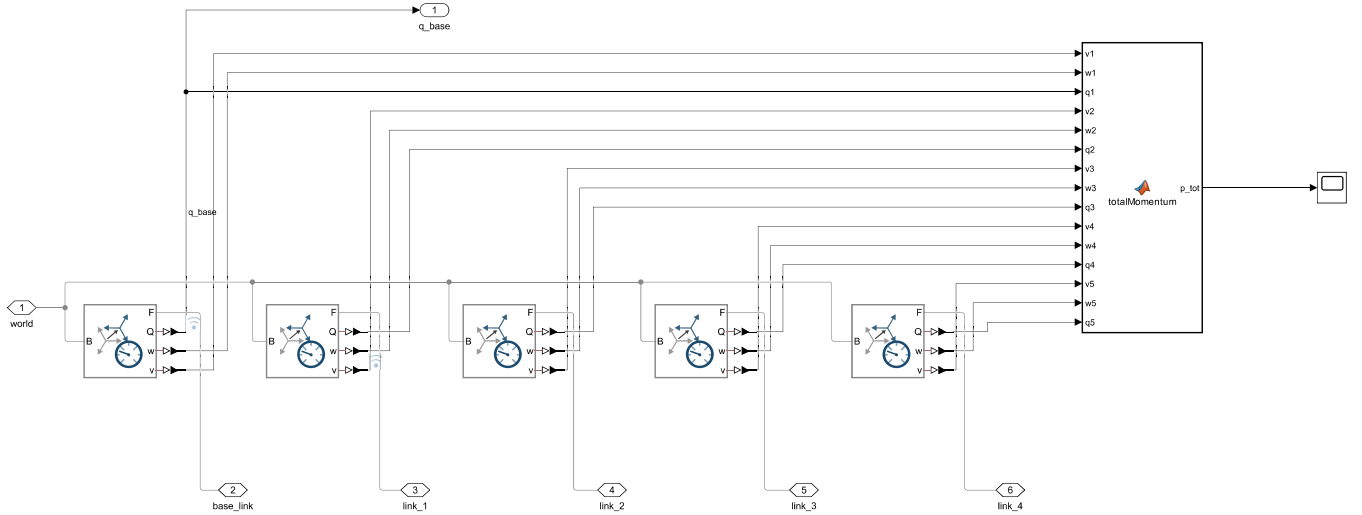


FIGURE 6: Momentum monitor subsystem (momentum observer) of the Simulink model `SpaceRobot.slx`. The velocities and orientations of the base and of all manipulator links are measured and combined in a MATLAB function to compute the total linear momentum of the free-floating space robot. The monitor is used to check momentum conservation in the simulation.

not influence the training.

Collision Monitor. At every agent step, a collision monitor computes the minimum distance $d_{\min}$ between non-adjacent robot bodies from the URDF collision geometry (see Fig. 8) [39]. If $d_{\min}$ falls below the safety threshold $d_{\text{safe}} = 2$ cm, a warning is raised and logged. If a contact between two bodies is detected, all commanded joint torques are overridden to zero and the episode is terminated with the penalty in (5). The monitor was tested by intentionally commanding the arm toward the base and by starting an episode in a colliding configuration. In both cases, the contact was detected, and the robot was stopped (see Fig. 9). During training, collisions occurred in early exploratory episodes for some agents, but were captured by the monitor and penalized through episode termination. In the final evaluation, no collisions were observed for any agent ($K_5 = 0$ over all 30

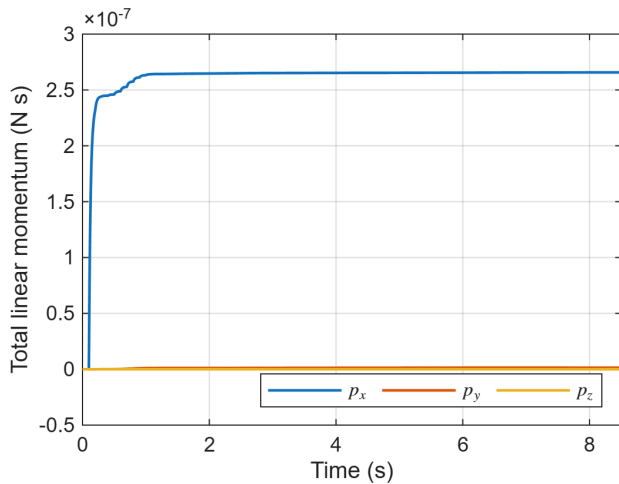


FIGURE 7: Total linear momentum of the robot during an episode.

evaluation episodes).

Joint Limit Enforcement. Each revolute joint of the arm is subject to mechanical limits. Violating these limits could damage a real robot and often indicates an unstable controller. The joint limits are enforced at two levels. First, the Simscape model includes mechanical joint limits with penalty forces, i.e., a stiff barrier that opposes further motion beyond the limit. Second, a joint-limit monitor in the reward block compares every joint angle with its admissible range at each agent step. A violation terminates the episode with the penalty in (5), which prevents further motion into the limit. After training, the joint-limit violation rate is low. PPO and TRPO reach the limits in a few percent of evaluation steps (5.1% and 2.1%, respectively), whereas TD3, SAC, PG, and DDPG remain within the admissible range ($K_6 \approx 0$). The higher K_6 value of PPO ($K_6 = 0.0505$) coincides with the lowest tracking error and may indicate a more extensive use of the workspace. In all cases, the enforcement mechanism stopped further simulated motion at the limit. The torques were set to zero in the offending step and the episode was marked as failed. For the optimized PPO agent (Section VII), no such events are observed in the evaluation episodes ($K_6 = 0$).

Formal Verification of Torque Constraints. In addition to runtime saturation, an offline formal verification was performed to check torque-bound compliance of the trained policy block. Using MATLAB Simulink Design Verifier (SLDV) in property-proving mode, a simplified closed-loop model of the agent and the joint torques was analyzed. The property to be checked is that, for any admissible observation input in the analyzed abstraction, each commanded torque τ_i satisfies $\tau_i \in [-\tau_{\max}, +\tau_{\max}]$ (see Fig. 10). SLDV found no counterexample for this property. The corresponding SLDV result is shown in Fig. 11 [23]. This result applies to the

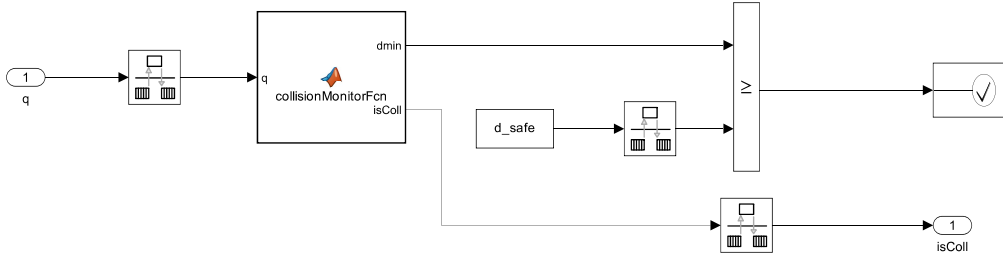


FIGURE 8: Collision monitor subsystem of the Simulink model `SpaceRobot.slx`. The joint and link configurations are used to compute the minimum distance between non-adjacent robot bodies. A distance below the safety threshold raises a warning. A detected contact sets the collision flag, which terminates the episode.

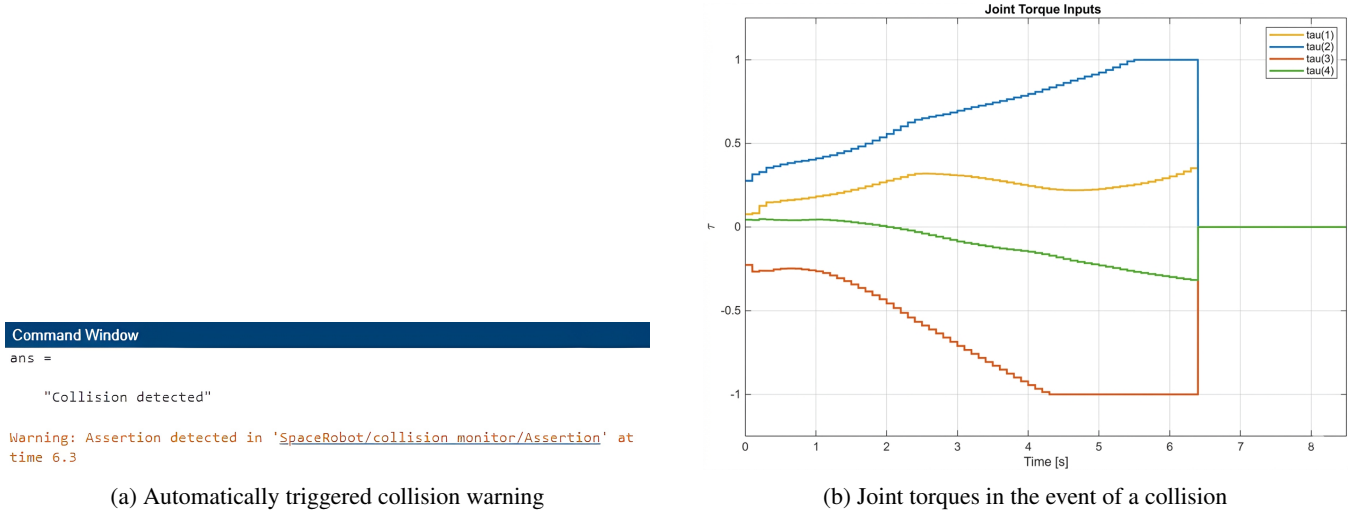


FIGURE 9: Test of the collision monitor and the associated joint torques.

simplified verification model, including admissible abstract states not visited during simulation. The full Simscape model is not amenable to property proving because of the nonlinear dynamics. The verification was performed on a simplified model (`Verify_tau.slx`) in which the plant was replaced by a worst-case abstraction, because only the policy output block is relevant for the considered property. This illustrates how formal methods can complement simulation-based learning by checking selected properties on an abstracted model of the agent.

Limitations of the Formal Methods Component. The mechanisms above cover a defined set of monitored properties, namely collision events, joint-limit events, bounded torques, and physical consistency through the momentum check. Other properties, such as task completion within a time bound or formal stability margins, are not verified. In particular, the runtime monitors detect violations and terminate the episode but do not prove their absence, and the offline SLDV check establishes only the torque bound on the simplified verification model. Neither closed-loop stability, task completion, nor collision avoidance over all possible trajectories of the full nonlinear Simscape model is formally verified. A complete formal specification of the mission, for example temporal-logic specifications such as

“the robot shall accomplish X without Y happening”, would be considerably more complex and is beyond the scope of this work. This work targets the safety properties that are considered most relevant for the task. Within this scope, the runtime monitors terminated episodes that would have violated a monitored property and penalized the corresponding actions. By the end of training, no violations of the monitored constraints were observed in the reported evaluation episodes [25]. This is empirical evidence for the tested initial configuration and reference trajectories, not a guarantee over all reachable states.

Computational Overhead and Scalability. The momentum observer and the joint-limit monitor consist of simple algebraic operations over the link states and scale as $\mathcal{O}(n_{\text{links}})$ per time step. The pairwise distance check of the collision monitor scales as $\mathcal{O}(n_{\text{links}}^2)$. For the 4-DOF arm considered here, this overhead is small compared with the physics simulation itself. End-to-end training times, including the full safety stack, are reported as metric T_2 in Table 7. They range from about 7 minutes (PG) to 35 minutes (SAC) for 1000 episodes on a standard workstation (AMD Ryzen 7 7700, 32 GB RAM), with six training runs executed in parallel. For onboard deployment, only the learned policy network needs to execute in the control loop. The default and the optimized

PPO agents use a compact policy network with two hidden layers of 128 units and about 2.1×10^4 parameters. This is well below the memory footprint of typical perception networks deployed on space-qualified hardware. At the 10 Hz control rate used here, a single forward pass through this network requires about 2×10^4 multiply-accumulate operations per step. This load appears compatible with the throughput of modern space-qualified embedded processors, for example ARM Cortex-A or LEON-class CPUs with a hardware floating-point unit. The safety monitors can be retained as lightweight parallel watchdog processes or reduced to the most safety-critical checks (torque saturation and joint-limit monitor) with little additional computation. This modular structure may simplify the transfer to resource-constrained onboard systems.

In summary, the runtime safety monitors integrated into the learning process terminated unsafe simulated states during the exploratory phase of training and shaped the agent's behavior by penalizing unsafe actions. The agent therefore learns under runtime monitoring of the selected constraints. This combination of monitored runtime constraints and penalty-driven learning is one approach to training deep RL policies in safety-critical domains such as space robotics, in which unconstrained trial-and-error exploration is not admissible. The next section analyzes the performance of the six RL agents under these conditions.

VI. COMPARISON OF CONTINUOUS RL AGENTS

This section compares six continuous-action RL algorithms from three families on the same task and environment [22],

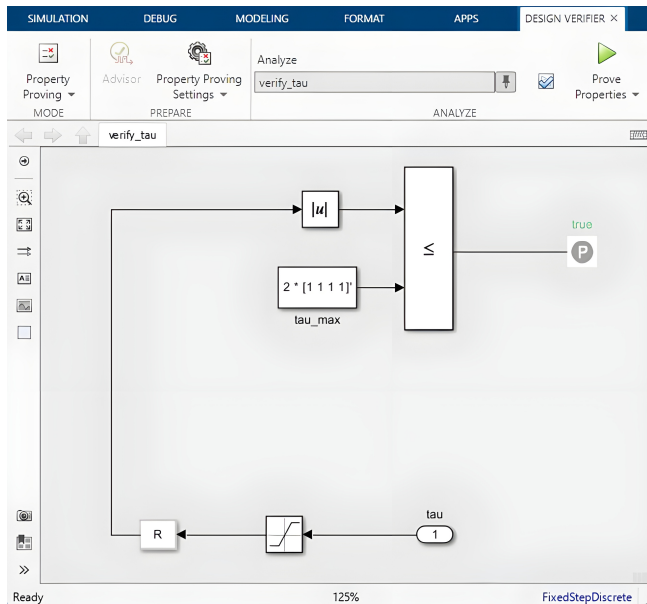


FIGURE 10: Property model for Simulink Design Verifier (Verify_tau.slx). The absolute joint torques are compared with the torque limit using a relational operator. The property is checked over the analyzed abstract state space of the simplified verification model.

[30]. The following algorithms are considered:

- *Policy Gradient (PG)*. The REINFORCE algorithm (Monte Carlo policy gradient), included as a baseline for direct policy optimization without variance-reduction techniques [21], [22].
- *Proximal Policy Optimization (PPO)*. A policy-gradient method that uses a clipped probability-ratio surrogate to bound policy updates [20].
- *Trust Region Policy Optimization (TRPO)*. A policy-gradient method that enforces a trust-region constraint based on the Kullback–Leibler divergence at each update. The constrained optimization step incurs a higher per-update computational cost than PPO [17].
- *Deep Deterministic Policy Gradient (DDPG)*. An off-policy actor-critic algorithm with a deterministic policy and target networks. DDPG jointly learns a Q -function and a policy and is sensitive to hyperparameter choices [43].
- *Twin Delayed DDPG (TD3)*. A variant of DDPG that uses two critic networks to reduce overestimation bias and delays policy updates relative to value updates [18].
- *Soft Actor-Critic (SAC)*. An off-policy actor-critic algorithm with a stochastic policy and an entropy term in the objective. This entropy regularization promotes exploration but increases the per-step computational cost relative to deterministic methods [19].

The six algorithms cover the on-policy/off-policy and deterministic/stochastic axes, as well as a range of update rules. Given the strong base-manipulator coupling and the safety constraints, methods with constrained or clipped policy updates (PPO, TRPO) are expected to provide more stable training than algorithms without such constraints (PG, DDPG). The off-policy actor-critic methods (DDPG, TD3, SAC) may achieve comparable performance but are typically more sensitive to hyperparameter selection.

On-policy methods (PG, PPO, TRPO) update the policy

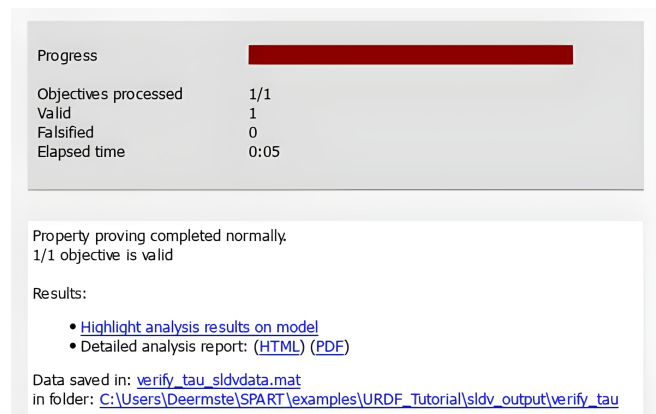


FIGURE 11: Result of the Simulink Design Verifier for the torque-bound property. The analysis found no counterexample for any analyzed execution path of the simplified verification model. The result does not cover other properties or the full nonlinear Simscape model.

TABLE 5: Classification of the evaluated reinforcement learning algorithms.

Algorithm	Policy Type
Policy Gradient (PG)	On-policy
Proximal Policy Optimization (PPO)	On-policy
Trust Region Policy Optimization (TRPO)	On-policy
Deep Deterministic Policy Gradient (DDPG)	Off-policy
Twin Delayed DDPG (TD3)	Off-policy
Soft Actor-Critic (SAC)	Off-policy

only from data generated by the current policy, which typically leads to stable but less sample-efficient learning. Off-policy methods (DDPG, TD3, SAC) reuse experience stored in a replay buffer, which improves sample efficiency at the cost of higher sensitivity to hyperparameters [22], [30].

Training Setup. All experiments were performed on a workstation with an AMD Ryzen 7 7700 processor and 32 GB of RAM, using MATLAB R2026a with the Reinforcement Learning Toolbox and Simscape Multibody. All six agents use the default hyperparameters and the default networks of the Reinforcement Learning Toolbox with two hidden layers of 128 ReLU units. Only the sample time is set to Δt . Table 6 lists the resulting values. The reward function, the network size, and the training budget are the same for all agents, and no agent is tuned individually. Observed differences therefore reflect the algorithms together with their default hyperparameters. PPO was further tuned in a subsequent step. In the main comparison, PPO uses its default configuration. Each configuration was trained with ten seeds for 1000 episodes, and the policy after the last episode was evaluated.

Performance Metrics. The agents were evaluated on the KPIs introduced in Section IV with the protocol described there, which uses 30 random initial states for each of the ten seeds. Table 7 reports a subset of these metrics for the six agents.

The results in Table 7 show several patterns. With respect to training stability, TRPO and PPO yield the smallest late-training reward variance ($T_1 = 0.61$ and $T_1 = 1.78$, respectively), whereas DDPG shows the largest ($T_1 = 64.5$). The training time T_2 is shortest for PPO (≈ 8 min). The off-policy methods are slower (TD3 ≈ 38 min, SAC ≈ 35 min).

For trajectory tracking, PPO attains the lowest mean squared error ($K_2 = 0.0257$), lower than that of TRPO ($K_2 = 0.0680$). DDPG attains the lowest peak error ($K_3 = 0.4758$) but with large variability across runs. PG, SAC, and TD3 yield larger tracking errors. The mean error of PG is approximately six times that of PPO.

For base-motion minimization, PPO and TRPO yield comparable base-orientation errors ($K_4 = 0.0667$ and $K_4 = 0.0694$), while DDPG produces a larger drift ($K_4 = 0.2682$). TD3 yields a low base angular velocity ($K_8 \approx 0.02$ rad/s) at the cost of degraded tracking accuracy, consistent with a conservative policy that suppresses actuation. No collisions were observed for any agent ($K_5 = 0$). PPO and TRPO approach the joint limits in a small fraction of the steps

($K_6 = 5.1\%$ and $K_6 = 2.1\%$, respectively), corresponding to a wider use of the joint range.

PPO produces the smoothest control signals ($K_7 = 0.6812$). The highest jerk values are observed for TD3 and DDPG ($K_7 \approx 0.8$ – 1.0). The energy consumption K_9 is moderate for PPO (≈ 6.33) and lowest for TRPO (≈ 3.76). The low values for DDPG and TD3 ($K_9 < 1.0$) coincide with insufficient actuation and should not be interpreted as energy efficiency. Qualitatively, PPO traces the circular reference with low base disturbance. TRPO produces a similar profile with a slower response. SAC oscillates around the reference. TD3 adopts a conservative strategy that reduces tracking accuracy. DDPG is inconsistent across runs, in line with the large T_1 . PG does not reach reliable tracking within 1000 episodes.

Summary of the Comparison. The comparison can be summarized as follows:

- PPO shows the most favorable combination of training stability (low T_1), training time (T_2), tracking accuracy (low K_2), and control smoothness (low K_7) in this comparison, at the cost of a moderate joint-limit violation rate.
- TRPO is close to PPO on most metrics, but requires more computation time and still incurs occasional joint-limit violations.
- The off-policy methods (TD3, SAC) do not improve on PPO under unified conditions and are more computation-intensive, with higher tracking errors.
- PG and DDPG show the largest fluctuations and the weakest performance on most KPIs.

These results are consistent with the on-policy/off-policy distinction in Table 5. Under the strong dynamic coupling and the safety constraints of the considered task, the on-policy methods with constrained updates were the most reliable in this benchmark. Based on these results, PPO is selected for further tuning.

The next section reports the hyperparameter modifications applied to PPO and their effect on the KPIs.

Statistical Significance of Agent Comparisons. To test the ranking above statistically, non-parametric hypothesis tests were applied to the 30 evaluation episodes per agent. First, a Friedman test indicated that at least one agent differs significantly from the others for every KPI (all $p < 10^{-17}$). Pairwise Wilcoxon signed-rank tests were then conducted between PPO (baseline) and each competing agent. A Holm correction was applied to control the family-wise error rate across the multiple comparisons. Table 8 reports the Holm-corrected p -values and the median differences. Positive values indicate a better result of PPO for a minimization objective, and negative values indicate a better result of PPO for a maximization objective.

The statistical tests support the conclusions drawn from the summary metrics and add some details. For episodic return (K_1), TRPO and TD3 achieve slightly higher returns than PPO ($\Delta\hat{x} = +7.55$ and $+8.90$, respectively), but both converge to a conservative strategy. TD3 in particular

TABLE 6: Hyperparameters of the benchmarked agents. The six default agents use the default values of the Reinforcement Learning Toolbox (R2026a). All agents act every $\Delta t = 0.1$ s and use networks with two hidden layers of 128 ReLU units. α_a and α_c denote the actor and critic learning rates.

Agent	α_a	α_c	γ	Experience	Mini-batch	Further settings
PG	10^{-2}	10^{-2}	0.99	1 episode	–	entropy weight 0, critic as baseline
PPO (default)	10^{-2}	10^{-2}	0.99	horizon 512	128	3 epochs, clip 0.2, entropy 0.01, GAE 0.95
TRPO	–	10^{-2}	0.99	horizon 512	128	KL limit 0.01, entropy 0.01, GAE 0.95
DDPG	10^{-2}	10^{-2}	0.99	buffer 10^4	64	target smoothing 0.001, OU noise $\sigma = 0.3$
TD3	10^{-2}	10^{-2}	0.99	buffer 10^4	64	target smoothing 0.005, noise $\sigma = 0.32$, policy delay 2
SAC	10^{-2}	10^{-2}	0.99	buffer 10^4	64	target smoothing 0.001
PPO (optimized)	10^{-3}	5×10^{-4}	0.995	horizon 1024	256	10 epochs, clip 0.2, entropy 0.001, GAE 0.95, gradient threshold 1

TABLE 7: Selected performance metrics of each RL agent. The best value in each row is shown in bold. PPO attains the lowest mean squared tracking error (K_2), the lowest base orientation error (K_4), and the lowest jerk (K_7), with the shortest training time (T_2). DDPG attains the lowest maximum error (K_3). TRPO is close to PPO on several metrics but requires more training time. The off-policy methods (TD3, SAC, DDPG) show larger errors or a larger late-training reward variance, for example a high T_1 for DDPG. Values are averaged over 30 evaluation episodes per agent.

KPI (unit)	TRPO	TD3	SAC	PPO	PG	DDPG
K_2 : Mean Squared EE Position Error	0.0680	0.1914	0.1908	0.0257	0.1638	0.0937
K_3 : Maximum EE Position Error	0.6783	0.5341	0.7766	0.4833	1.0654	0.4758
K_4 : Mean Base Orientation Error	0.0694	0.0868	0.1687	0.0667	0.1302	0.2682
K_6 : Joint Limit Violation Rate	0.0214 (2.1%)	0	0	0.0505 (5.1%)	0	0
K_7 : Control Smoothness (Jerk)	0.7498	0.9981	0.7536	0.6812	0.7125	0.8118
T_1 : Reward Std. (Late Training)	0.6144	8.5168	9.6907	1.7756	13.1970	64.4764
T_2 : Training Time (1000 Episodes)	~13 min	~38 min	~35 min	~8 min	~8 min	~24 min

TABLE 8: Pairwise Wilcoxon signed-rank tests of each agent vs. PPO (baseline), Holm-corrected. Each cell shows $\Delta\tilde{x}$ (median difference, agent–PPO) with the significance level as superscript ($***p < 0.001$, $**p < 0.01$, $*p < 0.05$, $n.s.p \geq 0.05$). For K_1 , positive $\Delta\tilde{x}$ means the agent achieves a higher (better) return. For K_2 – K_9 , negative $\Delta\tilde{x}$ means lower (better) error/violation/jerk/energy values. K_5 is the collision rate (0 = no collisions). All evaluated agents achieved $K_5 = 0$.

Agent	K_1	K_2	K_3	K_4	K_5	K_6	K_7	K_8	K_9
TRPO	+7.55*	−0.277***	−0.526***	+0.003 ^{n.s.}	0 ^{n.s.}	−0.137***	+0.362***	−0.041***	−9.38***
TD3	+8.90***	+0.047 ^{n.s.}	−0.276***	−0.097***	0 ^{n.s.}	−0.156***	+0.731***	−0.095***	−12.91***
SAC	−33.79***	+0.908***	+0.981***	+0.093**	0 ^{n.s.}	−0.059***	+0.404***	+0.032***	+2.44*
PG	−24.46*	−0.051 ^{n.s.}	−0.132 ^{n.s.}	+0.158**	0 ^{n.s.}	−0.156***	+0.199***	0.000 ^{n.s.}	−4.05***
DDPG	−102.02***	+0.060 ^{n.s.}	−0.095 ^{n.s.}	+0.331***	0 ^{n.s.}	−0.153***	+0.724***	−0.006 ^{n.s.}	−12.17***

yields $K_7 \approx 0.998$ and $K_9 < 1$, which indicates near-zero actuation rather than effective tracking. SAC, PG, and DDPG produce significantly lower returns than PPO ($p < 0.001$ each), with DDPG showing the largest deficit ($\Delta\tilde{x} = -102$).

For tracking accuracy (K_2 , K_3), PPO differs significantly from TRPO, TD3, and SAC (all $p < 0.001$). The differences from PG and DDPG are not significant at the $\alpha = 0.05$ level after Holm correction, which is likely due to the high variability of these agents. The median difference is largest for SAC (K_2 , $\Delta\tilde{x} = +0.908$), which is consistent with the oscillatory behavior of SAC.

Regarding base stability and safety (K_4 – K_6), the base orientation error of PPO differs significantly from that of TD3, SAC, PG, and DDPG (K_4 , all $p < 0.01$). The difference from TRPO is not significant ($p = 0.926$), which indicates a comparable base stability of the two on-policy methods.

For the collision-rate KPI (K_5), all evaluated agents achieve $K_5 = 0$ over the 30 evaluation episodes. The collision metric therefore does not distinguish the agents in this evaluation and is reported descriptively rather than used for ranking.

For control quality and efficiency (K_7 – K_9), PPO produces significantly smoother control signals than all competing agents (K_7 , all $p < 0.001$), with DDPG and TD3 showing the largest jerk deficits ($\Delta\tilde{x} = +0.724$ and $+0.731$). Energy consumption (K_9) is significantly lower for TRPO, TD3, and DDPG than for PPO. However, as noted for K_1 , these low values are likely due to reduced actuation and not to higher efficiency. They have to be interpreted together with the tracking errors.

Overall, the tests are consistent with a favorable balance of PPO across the KPIs in this comparison. The smoother control signals of PPO are significant against all other agents,

and its base orientation error does not differ significantly from that of TRPO. Fig. 12 shows the distribution of four selected KPIs over the 30 evaluation episodes as boxplots, including the spread and the outliers of each agent. Fig. 13 shows the corresponding 95% bootstrap confidence intervals for each agent and KPI.

VII. OPTIMIZATION OF THE PPO AGENT

Based on the comparison in Section VI, PPO is selected for further tuning. The PPO agent used so far corresponds to the default configuration of MATLAB's Reinforcement Learning Toolbox [42]. The modifications aim at faster convergence, lower tracking and base orientation errors, and the avoidance of the remaining joint-limit violations. The tuning concerns the hyperparameters, while the hidden layers of the networks are kept.

Network Architecture. The optimized agent keeps the hidden layers of the default PPO agent. The policy and the value network both use two hidden layers of 128 units with ReLU activations. In contrast to the default agent, the mean output of the Gaussian policy is not bounded by a tanh layer. The commanded torques are limited by the saturation block instead. The networks are defined explicitly rather than through the toolbox default constructor.

Hyperparameter Tuning. The configuration was determined in a preliminary coordinate-wise grid search and is kept fixed for all experiments reported here. Each hyperparameter was swept over a discrete set of candidate values while all others were held at their current best values. The configuration with the highest mean episodic reward over the final 50 of the $N_{\text{train}} = 1000$ training episodes was selected. In total, 16 training runs of 1000 episodes each were conducted during this tuning phase. The swept parameters, their candidate values, and the selected values are as follows:

- *Actor learning rate.* Candidates $\{1 \times 10^{-3}, 5 \times 10^{-4}\}$, selected 1×10^{-3} . Both candidates are an order of magnitude below the toolbox default of 10^{-2} , which reduces the size of the policy updates.
- *Critic learning rate.* Candidates $\{1 \times 10^{-3}, 5 \times 10^{-4}\}$, selected 5×10^{-4} , which gave the highest mean episodic reward in the search.
- *Entropy loss weight.* Candidates $\{1 \times 10^{-2}, 1 \times 10^{-3}\}$, selected 1×10^{-3} . A small entropy bonus is intended to prevent a premature convergence to a suboptimal deterministic policy while avoiding undirected exploration.
- *Experience horizon.* Candidates $\{512, 1024, 2048\}$, selected 1024. A moderate rollout horizon is a compromise between the trajectory information per update and the memory requirements and update latency.
- *Mini-batch size.* Candidates $\{128, 256\}$, selected 256. Larger mini-batches are expected to give more stable gradient estimates and thus a lower training variance.
- *Fixed settings.* The discount factor was set to $\gamma = 0.995$, the number of epochs per update to 10, and the gradient threshold of both optimizers to 1. The clip factor of

0.2 and the GAE factor of 0.95 remain at their default values.

In the tuning runs, the resulting configuration gave a more stable and better-performing PPO agent. Compared with the default configuration, the optimized agent uses learning rates that are ten to twenty times smaller, larger rollouts and mini-batches, more epochs per update, and gradient clipping. Together, these settings limit the size of each policy update.

A. CIRCULAR TRAJECTORY

Training Outcome. The effect of the modifications can be seen in the training curves (see Fig. 14). The episodic return of the default PPO agent increases initially and then plateaus around -2 per episode with high variance over the remaining training. This suggests a convergence to a suboptimal policy. In contrast, the return of the optimized PPO agent continues to increase and converges to a higher value (closer to zero) with reduced variance. Table 9 reports the KPI averages over $N_{\text{eval}} = 30$ evaluation episodes for PPO with the default and the optimized configurations. The mean episodic return K_1 increases from -2.23 for the default PPO agent to -0.43 for the optimized PPO agent, which is about five times closer to zero. The reduced variance indicates that the improvement is consistent across episodes rather than driven by a few outliers.

The higher return is reflected in the task-level KPIs. Fig. 15 shows the end-effector path in the XY plane. The default PPO agent deviates from the reference circle and oscillates, whereas the optimized PPO agent follows the reference closely. The mean squared tracking error K_2 decreases from 0.0257 to 0.0083 (a reduction of 67.7%), and the maximum error K_3 from 0.4833 to 0.27.

Base stabilization improves as well (Fig. 16). The mean orientation error K_4 decreases from 0.0667 to 0.022 (a factor of approximately three), and the mean angular velocity K_8 decreases from 0.095 rad/s to 0.050 rad/s. The lower K_4 suggests that the optimized agent partly compensates the momentum transfer to the base with its arm motion.

For the optimized PPO agent, no violations of the moni-

TABLE 9: Performance of the PPO agent before and after optimization. The optimized PPO agent improves most reported KPIs. It reaches a higher return, lower errors, and smoother control, with no observed violations of the monitored properties in the evaluation episodes.

Metric (KPI)	PPO (default)	PPO (optimized)
Mean Episodic Return (K_1)	-2.23	-0.43
Mean Squared EE Position Error (K_2)	0.0257	0.0083
Maximum EE Position Error (K_3)	0.4833	0.27
Mean Base Orientation Error (K_4)	0.0667	0.022
Collision Rate (K_5)	0	0
Joint Limit Violation Rate (K_6)	0.0505 (5.1%)	0
Control Smoothness (K_7 , Jerk)	0.6812	0.351
Mean Base Angular Velocity (K_8)	0.095	0.050
Energy Consumption (K_9)	6.325	5.314

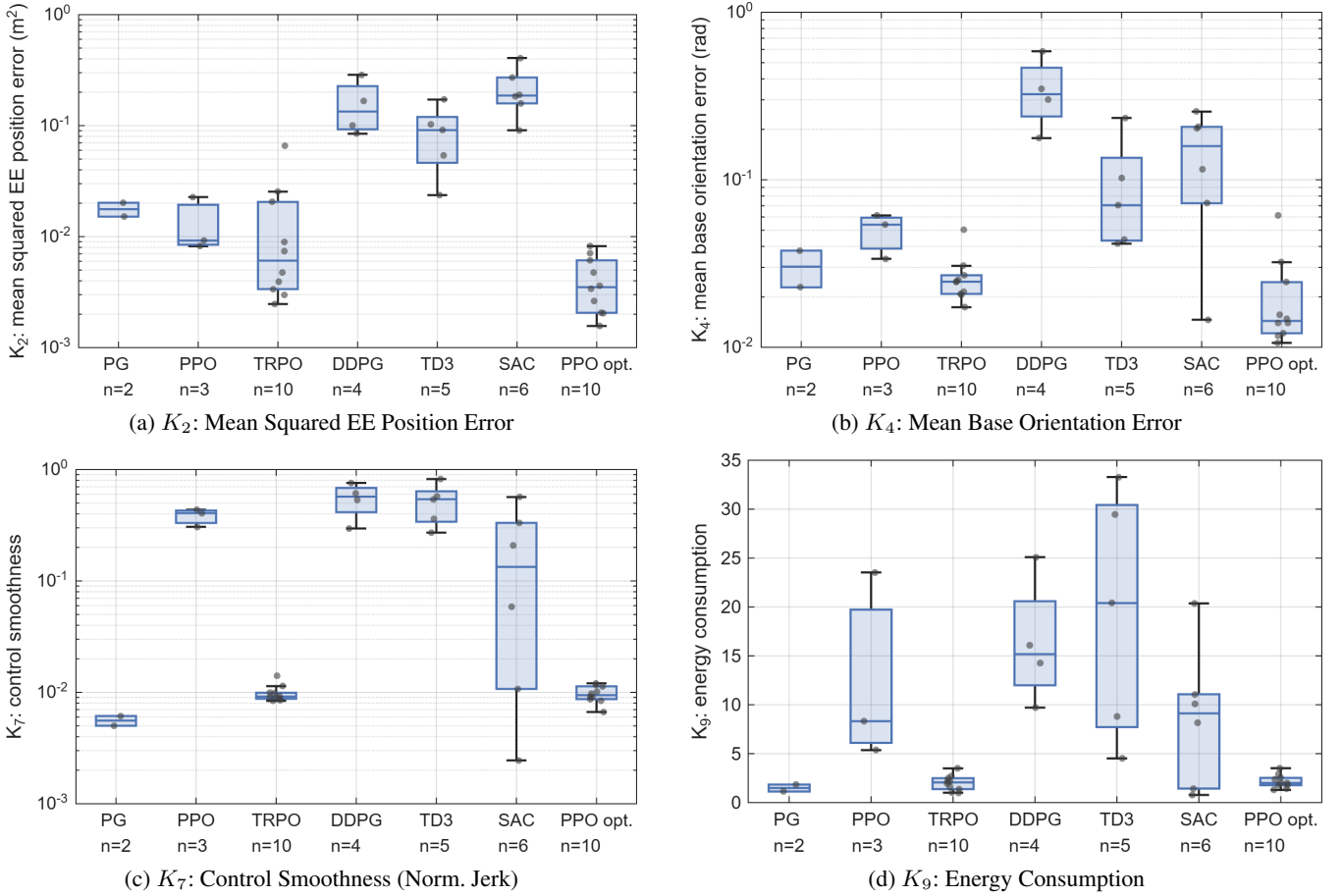


FIGURE 12: Boxplots of selected KPI distributions over 30 evaluation episodes per agent. Each box spans the interquartile range (IQR), the center line marks the median, the whiskers extend to $1.5 \times \text{IQR}$, and circles denote outliers. PPO shows a low tracking error (K_2) and smooth control (K_7) with a small spread, whereas PG and SAC vary more. The low K_9 values of TD3 and DDPG reflect insufficient actuation rather than energy efficiency.

tored properties are observed ($K_5 = K_6 = 0$ over the 30 evaluation episodes, compared to $K_6 = 5.1\%$ for the default configuration). It also improves the control smoothness (K_7 , $0.6812 \rightarrow 0.351$) and reduces the energy consumption by 16.0% (K_9 , $6.325 \rightarrow 5.314$). These changes are observed simultaneously in the reported evaluation, without an apparent trade-off among the selected KPIs.

In the reported evaluation, the optimized PPO agent improves tracking accuracy, base disturbance, control smoothness, and energy consumption together, without observed violations of the monitored constraints. These results support the use of hyperparameter tuning for space-robot control under the considered conditions.

B. ABLATION STUDY OF REWARD COMPONENTS

An ablation study quantifies the contribution of each term of the reward function in (1). The optimized PPO agent was retrained on the circular trajectory with one reward component disabled at a time. Each of the seven weights in the cost term c_t was set to zero in turn while the remaining six were kept at their baseline values ($W_p = 150$, $W_v = 25$,

$W_{\text{ori}} = 200$, $W_{\omega_b} = 8$, $W_{v_b} = 2$, $W_u = 0.02$, $W_d = 0.06$). In addition, the two positive reward terms were separately disabled by setting $r_{\text{prog}} = 0$ and $r_{\text{bonus}} = 0$, respectively, to isolate the contribution of the progress signal and the proximity bonus. All other aspects of the training and evaluation pipeline (hyperparameters, random seeds, episode horizon, and the $N_{\text{eval}} = 30$ evaluation protocol) were identical to the baseline configuration. The resulting KPIs are summarized in Table 10.

Dominant Weights (W_{ori} , W_p). Removing the base-orientation penalty has the largest effect. The mean episodic return K_1 drops from -0.43 to -23.65 , and the mean base orientation error K_4 rises by more than an order of magnitude (from 0.022 to 0.668). The end-effector tracking error ($K_2 = 0.017$) remains lower than in all other ablation variants. This suggests that without W_{ori} the agent focuses on end-effector tracking and largely loses the base stabilization. W_{ori} is therefore a central term of the reward and not only a fine-tuning term. Removing W_p degrades the end-effector tracking considerably. K_2 rises by a factor of about 29, K_3 by a factor of about 3.5 (not shown in the table), and K_1 drops

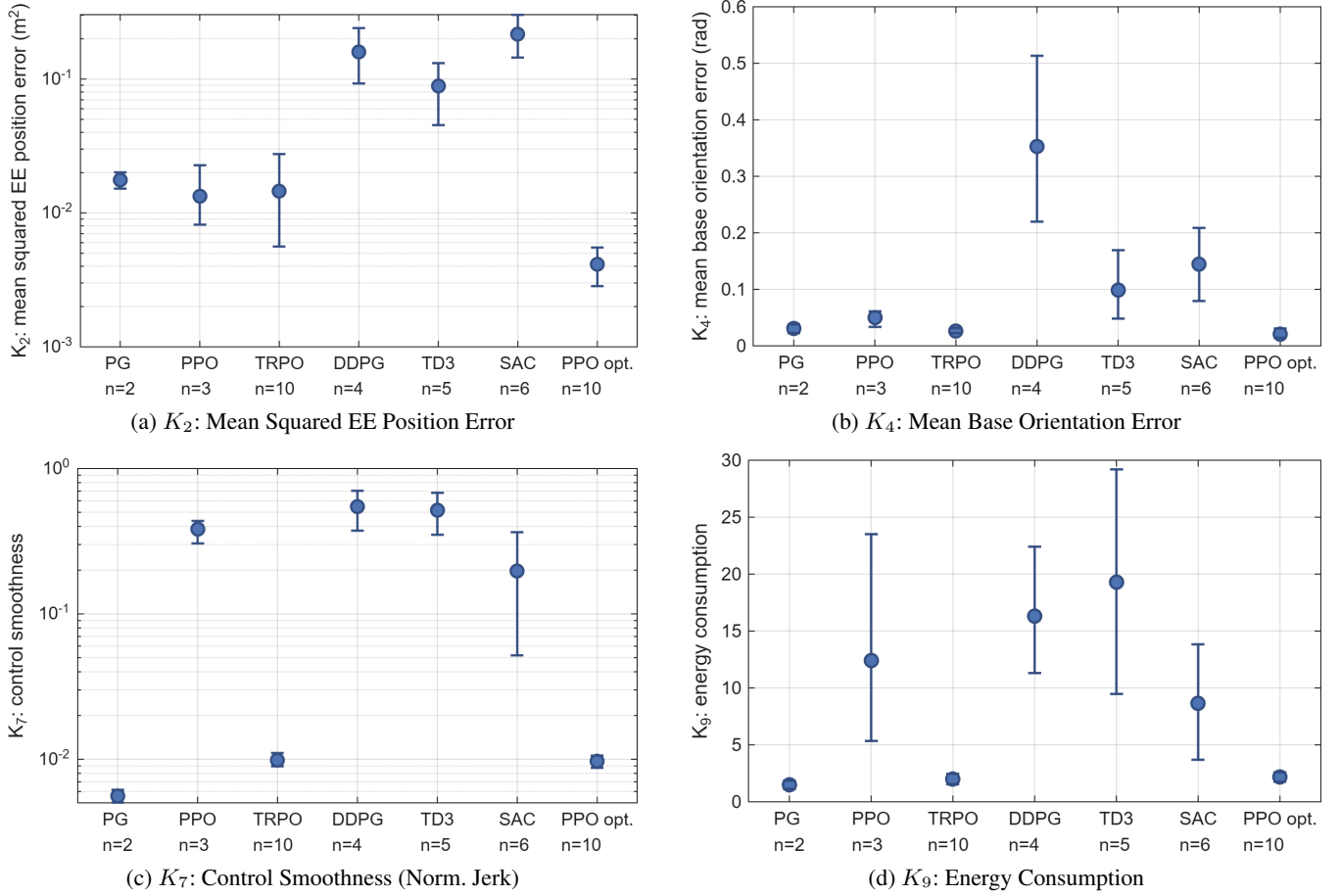


FIGURE 13: 95% bootstrap confidence intervals for selected KPIs across all agents (30 evaluation episodes per agent). The narrow intervals of TD3 and DDPG reflect a nearly constant behavior across episodes. The intervals of PPO are also narrow, although PPO uses a larger part of the workspace. TRPO and PPO show overlapping intervals for K_4 (base orientation), which is consistent with the non-significant Wilcoxon result ($p = 0.926$).

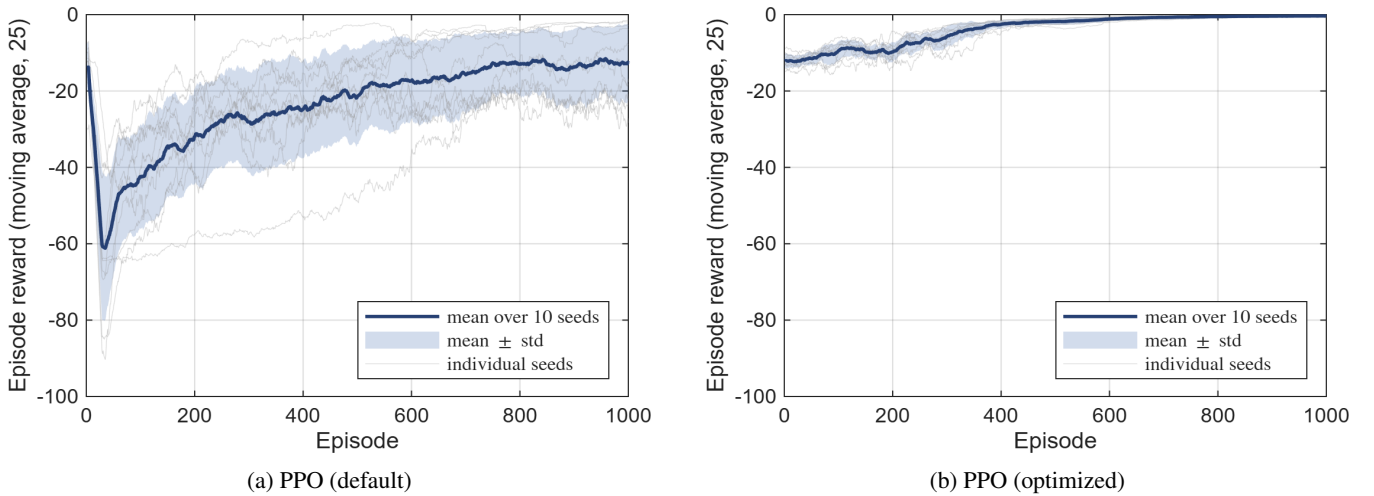


FIGURE 14: Episode rewards during training of the default and the optimized PPO agents. (a) Default hyperparameters. (b) Optimized configuration. The optimized configuration converges faster and reaches a higher average episodic return.

to -6.48 .

Secondary Weights ($W_v, W_{\omega_b}, W_{v_b}$). Each of these terms has a small but measurable effect. K_1 lies in the range

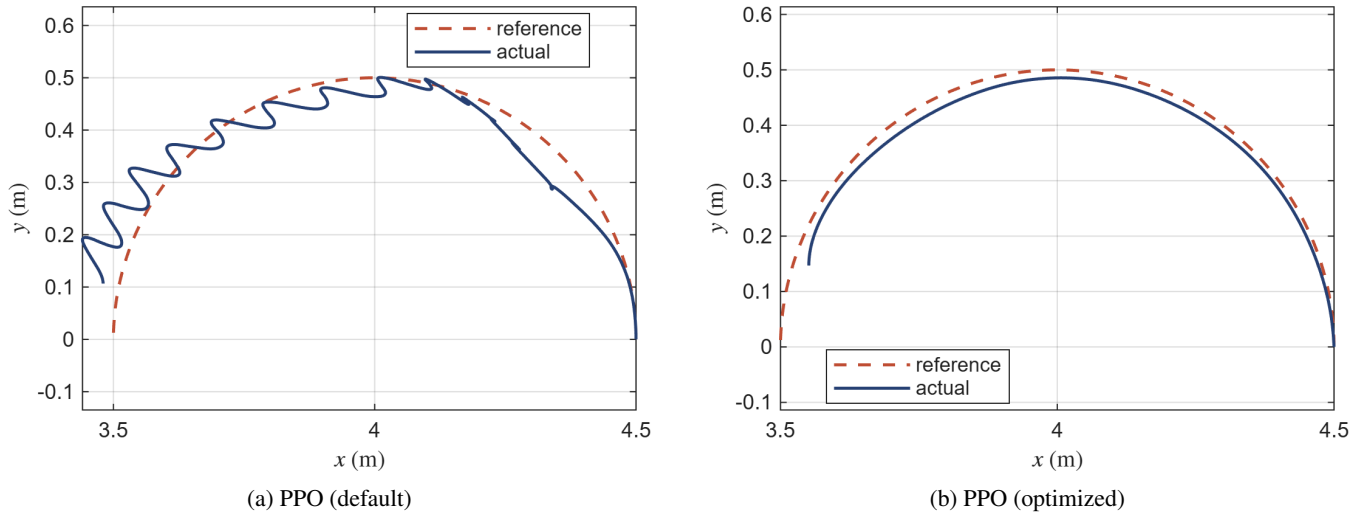
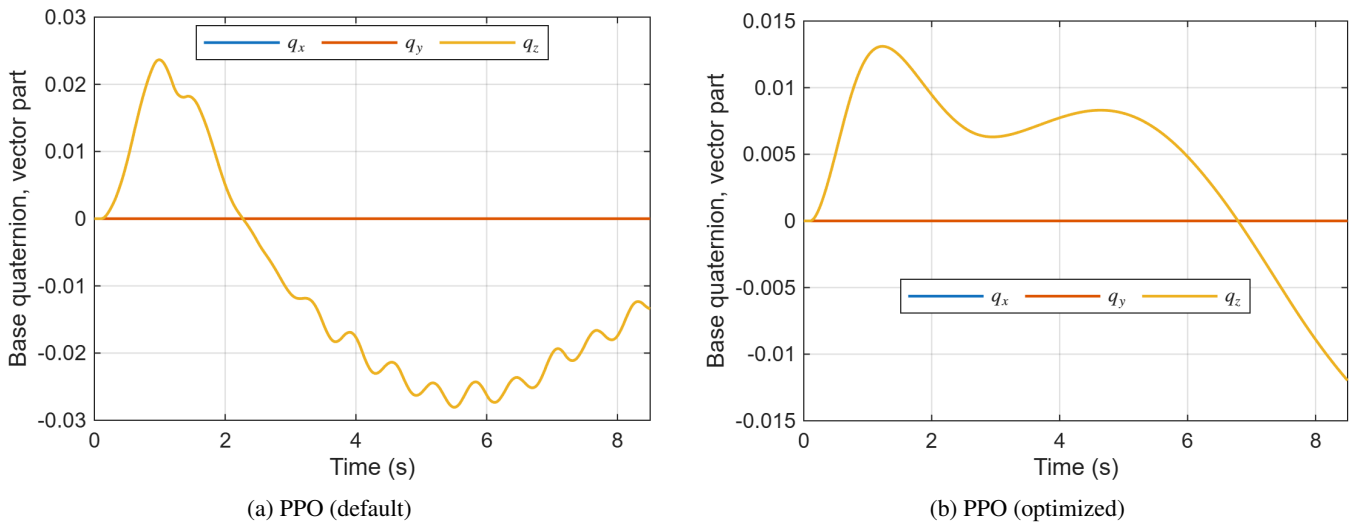
FIGURE 15: Reference and actual end-effector trajectory in the XY plane.

FIGURE 16: Time evolution of the base orientation (quaternion components) during a test episode.

TABLE 10: Ablation study of the reward components. Each row corresponds to a separately retrained optimized PPO agent with one reward weight set to zero. KPIs are averaged over $N_{\text{eval}} = 30$ evaluation episodes on the circular trajectory.

Variant	K_1	K_2	K_4	K_6	K_7	K_9
Baseline	-0.43	0.0083	0.022	0	0.351	5.314
$W_p = 0$	-6.48	0.238	0.046	0	0.626	2.868
$W_v = 0$	-0.85	0.022	0.049	0.0015	0.655	3.561
$W_{\text{ori}} = 0$	-23.65	0.017	0.668	0.047	0.588	4.363
$W_{\omega_b} = 0$	-1.17	0.032	0.059	0.0022	0.657	3.554
$W_{v_b} = 0$	-1.31	0.039	0.054	0.0004	0.670	3.192
$W_u = 0$	-1.32	0.037	0.060	0	0.618	3.856
$W_d = 0$	-1.10	0.029	0.057	0.0003	0.632	3.724
$r_{\text{prog}} = 0$	-3.65	0.075	0.049	0.0173	0.618	3.365
$r_{\text{bonus}} = 0$	-3.96	0.076	0.057	$3.8\text{e-}5$	0.627	4.078

$[-1.31, -0.85]$ compared with the baseline value of -0.43 . Tracking accuracy and smoothness are noticeably worse (K_2 rises by a factor of 2 to 5, and K_7 roughly doubles). These terms shape the motion quality and the damping of the base velocity, but none of them is decisive for task completion in this benchmark.

Control-Effort Weights (W_u, W_d). Disabling the penalties on torque magnitude and torque change moderately increases the tracking error (K_2) and roughly doubles the jerk metric K_7 , while K_1 remains between -1.32 and -1.10 . This suggests that these terms mainly shape the smoothness of the actuation and have little effect on task completion.

Positive Reward Terms ($r_{\text{prog}}, r_{\text{bonus}}$). Removing either of the two positive reward terms causes a clear drop in performance. K_1 falls to -3.65 and -3.96 , respectively, and the tracking error K_2 increases by roughly an order of magnitude (from 0.0083 to ≈ 0.075). Without r_{prog} , the

agent lacks the dense signal that rewards each reduction of the position error. The joint-limit violation rate K_6 rises to 1.73%, which suggests more aggressive motions without this step-wise feedback. Without r_{bonus} , the agent lacks the local attraction near the reference trajectory, which may explain the less accurate tracking. Both terms therefore have a clear effect on the final tracking accuracy in this ablation.

Note on the Energy Metric K_9 . K_9 is lower in every ablation variant than in the baseline (5.314). This does not indicate a higher energy efficiency. It is likely an artifact of earlier episode termination when the agent fails (for example, after a joint-limit violation or a contact), which truncates the integration window. K_9 should therefore only be interpreted together with K_1 , K_2 , and K_4 .

Summary. The ablation indicates that W_{ori} and W_p are the two dominant cost components of the reward. The remaining five cost weights refine the motion quality, the damping of the base velocity, and the smoothness of the actuation. Both positive reward terms (r_{prog} and r_{bonus}) also affect the final tracking accuracy. Removing a single secondary cost weight still yields a working, if degraded, policy. Removing W_{ori} , W_p , or one of the positive reward terms leads to a markedly degraded controller. This supports the reward design in Section IV and suggests that the results of the optimized PPO agent do not depend on a fine balance of the low-weight terms.

C. SENSITIVITY ANALYSIS OF REWARD WEIGHTS

The ablation study in Section VII-B identifies which reward components are most influential. A complementary question is how sensitive the results are to a moderate miscalibration of the reward weights. The analysis examines whether the PPO agent still converges to a working policy when the cost weights are perturbed and whether the ranking of the algorithms in Table 7 is likely to change. A one-at-a-time sensitivity analysis was performed on four cost weights. These are the two dominant weights W_p and W_{ori} from the ablation and the two secondary weights W_v and W_{ω_b} . Each weight was multiplied by 2 and by 0.5 around its baseline value ($W_p = 150$, $W_v = 25$, $W_{\text{ori}} = 200$, $W_{\omega_b} = 8$). All other settings were kept at the baseline. These are the remaining reward weights, the optimized PPO hyperparameters (Section VII), the random seeds, the episode horizon, and the evaluation protocol with $N_{\text{eval}} = 30$. This yields eight retrained policies, whose KPIs are summarized in Table 11.

Dominance of W_{ori} . The base-orientation weight shows the largest sensitivity. K_1 varies from -2.82 at $W_{\text{ori}} \times 0.5$ to -6.89 at $W_{\text{ori}} \times 2$, a spread of more than a factor of two. Doubling W_{ori} shifts the balance toward base stabilization at the cost of end-effector tracking. K_2 rises to 0.222, about 27 times the baseline value, while K_4 is 0.041. Halving W_{ori} relaxes the base stabilization, and the task is still solved. The agent still tracks the reference ($K_2 = 0.0438$), but with about twice the baseline base orientation error ($K_4 = 0.050$ vs. 0.022). This is consistent with the ablation result that W_{ori} is the dominant cost weight.

TABLE 11: One-at-a-time sensitivity analysis of four reward weights. Each row corresponds to a separately retrained optimized PPO agent with one weight multiplied by 2 or 0.5 around its baseline value. All other weights and training settings are held constant. KPIs are averaged over $N_{\text{eval}} = 30$ evaluation episodes on the circular trajectory.

Variant	K_1	K_2	K_3	K_4	K_7	K_9
Baseline	-0.43	0.0083	–	0.022	0.351	5.314
$W_p \times 2$	-4.56	0.0348	0.629	0.073	0.657	3.762
$W_p \times 0.5$	-3.62	0.0494	0.583	0.059	0.611	3.800
$W_v \times 2$	-3.07	0.0246	0.480	0.061	0.634	4.212
$W_v \times 0.5$	-3.48	0.0315	0.535	0.064	0.622	3.827
$W_{\text{ori}} \times 2$	-6.89	0.2221	0.907	0.041	0.622	3.059
$W_{\text{ori}} \times 0.5$	-2.82	0.0438	0.786	0.050	0.639	3.546
$W_{\omega_b} \times 2$	-3.41	0.0789	0.746	0.044	0.640	3.283
$W_{\omega_b} \times 0.5$	-3.00	0.0644	0.740	0.045	0.638	3.557

Secondary Dominance of W_p . The end-effector position weight is moderately sensitive. K_1 ranges from -3.62 to -4.56 , and K_2 rises by a factor of 4 to 6 over the baseline in both directions. Both perturbations disturb the balance between end-effector tracking and base stabilization, but neither causes the policy to fail. This is again consistent with the ablation, in which W_p is the second most influential weight.

Low Sensitivity of W_v and W_{ω_b} . The velocity-error and base-angular-velocity weights show the smallest K_1 spread, roughly symmetric around $K_1 \approx -3.2$. This suggests that these terms shape the motion quality and the base damping but do not change whether the task is solved. It agrees with the ablation, in which removing either weight still gave a working but degraded controller.

Safety Metrics. Across all eight perturbed configurations, no collisions were observed ($K_5 = 0$), and the joint-limit violation rate stayed below 0.53% ($K_6 \leq 0.0053$). In the tested cases, a miscalibration of the reward weights mainly degraded the task performance. The runtime monitors of Section V terminate episodes with violations independently of the reward weights.

Effect on the Algorithm Ranking. Across all eight weight perturbations, the optimized PPO agent still solves the task, with K_1 in $[-6.89, -2.82]$, $K_5 = 0$, and few joint-limit violations. In the configuration with the largest degradation ($W_{\text{ori}} \times 2$), the end-effector tracking error ($K_2 = 0.222$) is of the same order of magnitude as that of TD3 and SAC at the nominal weights ($K_2 \approx 0.19$ in Table 7). The other benchmarked agents (TRPO, SAC, TD3, PG, DDPG) are already noticeably weaker at the nominal reward weights. A perturbation of a single weight by a factor of 2 or 0.5 is therefore unlikely to change the ranking in Section VI. This argument is indirect, because only the PPO agent was retrained with perturbed weights. A full cross-evaluation of all six algorithms over the eight configurations would multiply the training budget by six and is beyond the scope of this benchmark.

Note on the Energy Metric K_9 . As in the ablation, K_9 is lower than the baseline value (5.314) in all eight variants. This likely reflects earlier episode termination in configurations where the agent fails to stabilize, which shortens the integration window. It does not indicate a higher energy efficiency. K_9 should therefore be interpreted together with K_1 , K_2 , and K_4 .

Summary. The sensitivity analysis supports the reward design in Section IV. W_{ori} is the most sensitive weight, W_p is second, and W_v and W_{ω_b} are comparatively insensitive. In the tested range from 0.5 to 2 times the baseline value, the optimized PPO agent solved the task for every tested weight, and no collisions were observed. The ranking in Table 7 is unlikely to change under a moderate miscalibration of a single weight, although this was only tested for the PPO agent.

D. ROBUSTNESS EVALUATION UNDER NON-IDEAL CONDITIONS

To evaluate the robustness of the optimized PPO policy beyond the nominal simulation assumptions, an additional stress-test campaign was performed on the circular trajectory. The already trained optimized PPO policy was evaluated without retraining over $N_{\text{eval}} = 30$ episodes for each condition. Five non-ideal cases were considered, namely sensor noise, reduced actuator authority, an external torque disturbance, parameter uncertainty, and a combined stress case. Sensor noise was injected directly between the observation vector and the RL agent using the MATLAB function $\text{obs}_{\text{noisy}} = \text{obs} + 0.005 \text{randn}(\text{size}(\text{obs}))$, while the underlying plant dynamics remained unchanged. Actuator saturation was tested by reducing all torque limits to $0.75 \tau_{\text{max}}$. The external disturbance was implemented as an additive joint torque after the torque-saturation block, with $\tau_{\text{dist},1} = \tau_{\text{dist},2} = 2 \text{ N m}$ for $2.0 \leq t \leq 2.5 \text{ s}$ and zero disturbance otherwise. Parameter uncertainty was modeled by multiplying all masses, inertias, and joint damping coefficients by a factor of 1.5. The combined case applied all four perturbations simultaneously.

Robustness Results. No violations of the monitored properties occurred in any stress test. No collisions or joint-limit violations were observed in any condition ($K_5 = K_6 = 0$). The strongest degradation is observed for the combined stress case and for parameter uncertainty. This is plausible, because both change the effective dynamics seen by the fixed policy. In the combined case, no divergence and no monitor-triggered termination were observed. The mean base orientation error increased to $K_4 = 0.0447$ and the mean tracking error to $K_2 = 0.0078$. Across all non-ideal conditions, K_7 increases compared with the nominal optimized PPO result, which suggests that the policy compensates for the perturbations with less smooth torque profiles. The lower K_9 values under stress should not be interpreted as efficiency gains. They likely reflect the changed actuator limits, the modified dynamics, and the injected disturbances, which alter the mechanical-work calculation. Overall, the experiment

indicates that the performance of the optimized PPO policy degrades gradually under sensor noise, disturbances, parameter uncertainty, and reduced actuator authority, with no violations of the monitored constraints in the tested episodes. Each perturbation is tested at a single level and from a fixed initial configuration. These stress tests therefore do not replace a systematic robustness characterization.

E. PIECEWISE-LINEAR TRAJECTORY

The previous evaluations consider tracking of a smooth circular end-effector reference. To assess how the controller transfers to a different reference geometry, an additional evaluation is performed on a piecewise-linear trajectory. This case is relevant for on-orbit manipulation tasks in which approach and retreat motions are planned as straight-line segments rather than continuous curves.

Trajectory Definition. The linear reference uses the same circle parameters (center and radius) as the circular benchmark. The end-effector is commanded to move along two straight segments that connect three points on that circle, $P_0 = [c_x + r, c_y, c_z]$, $P_1 = [c_x, c_y + r, c_z]$, and $P_2 = [c_x - r, c_y, c_z]$. Using a period T and a switching time $t_1 = T/2$, the piecewise-linear reference position $p_{\text{ref}}(t)$ is

$$p_{\text{ref}}(t) = \begin{cases} P_0 + \frac{t}{t_1} (P_1 - P_0), & 0 \leq t \leq t_1, \\ P_1 + \frac{t-t_1}{T-t_1} (P_2 - P_1), & t_1 < t \leq T. \end{cases} \quad (6)$$

The reference is sampled at $T_s = 0.01 \text{ s}$, and the PPO agent acts every $\Delta t = 0.1 \text{ s}$ as in the other experiments.

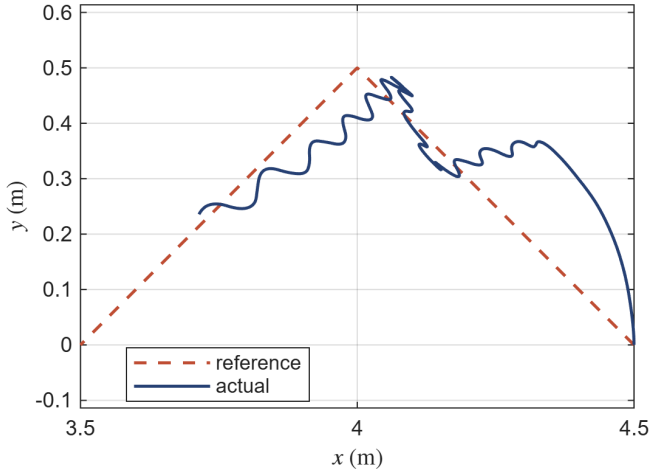
Results. Table 13 reports the KPI averages over $N_{\text{eval}} = 30$ evaluation episodes for PPO with the default and the optimized configurations on the piecewise-linear reference. The optimized PPO agent increases the episodic return (K_1 , $-0.4565 \rightarrow -0.2977$). It reduces the base attitude disturbance K_4 by approximately 44% (see Fig. 18), the residual base angular velocity K_8 by approximately 15%, and the energy consumption K_9 by approximately 35%. The mean and peak tracking errors (K_2 and K_3) remain close to those of the default configuration, with a small reduction of the mean error (see Fig. 17). The control smoothness metric K_7 , however, is worse than for the default PPO agent. This suggests that the hyperparameter configuration tuned for circular tracking does not fully transfer to piecewise-linear references. A possible mitigation is to include a mixture of reference types (circular, linear, and hybrid trajectories) during training or during the hyperparameter search.

Both evaluations support the use of hyperparameter tuning for space-robot control under the considered conditions. The piecewise-linear results also indicate that trajectory-specific tuning may be needed to maintain control smoothness across reference geometries.

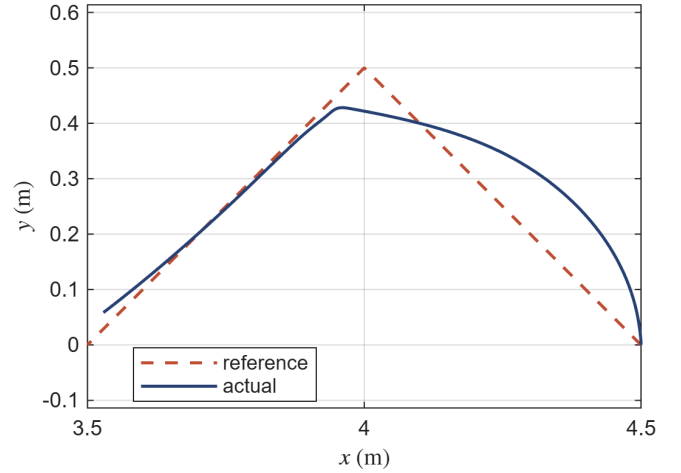
The following section discusses the implications, limitations, and future steps toward deployment on physical testbeds for space robotics.

TABLE 12: Robustness evaluation of the optimized PPO policy under non-ideal simulation conditions. The policy was evaluated without retraining over $N_{\text{eval}} = 30$ episodes per condition on the circular trajectory. K_1 is maximized, whereas K_2 – K_9 are minimized.

Condition	K_1	K_2	K_3	K_4	K_5	K_6	K_7	K_8	K_9
Nominal optimized PPO	−0.3709	0.0021	0.1614	0.0227	0	0	0.8401	0.0258	3.9425
Sensor noise	−0.4557	0.0030	0.2164	0.0227	0	0	0.6677	0.0398	3.8498
Reduced saturation ($0.75 \tau_{\text{max}}$)	−0.4300	0.0028	0.1982	0.0226	0	0	0.6912	0.0373	3.3913
External disturbance	−0.5791	0.0029	0.2052	0.0268	0	0	0.6671	0.0447	3.7735
Parameter uncertainty ($\times 1.5$)	−0.7728	0.0067	0.1775	0.0300	0	0	0.6786	0.0343	2.8879
Combined stress case	−1.3432	0.0078	0.2269	0.0447	0	0	0.6925	0.0402	2.4867

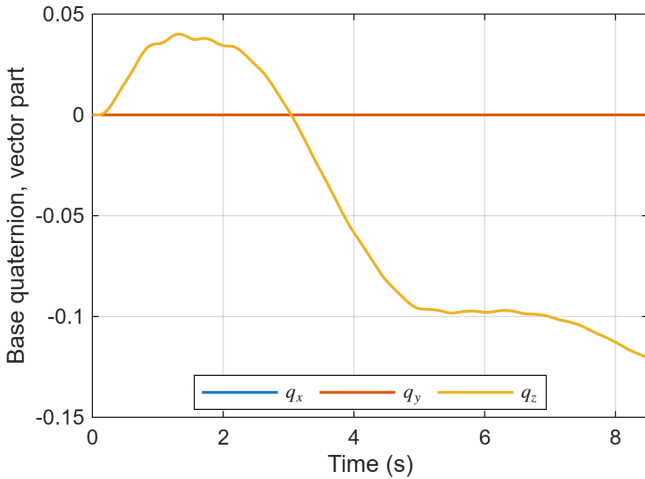


(a) PPO (default)

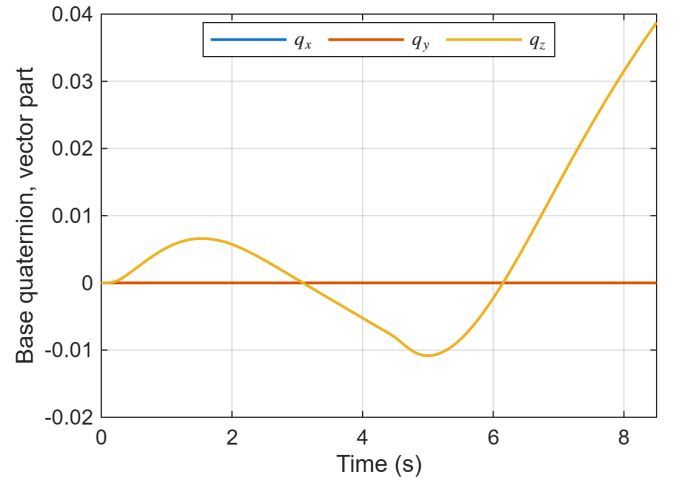


(b) PPO (optimized)

FIGURE 17: Reference and actual piecewise-linear end-effector trajectory in the XY plane.



(a) PPO (default)



(b) PPO (optimized)

FIGURE 18: Time evolution of the base orientation (quaternion components) during a test episode with a piecewise-linear trajectory.

VIII. DISCUSSION AND CONCLUSION

This work investigated continuous-action deep RL for the control of a free-floating space robotic arm, with a focus on control performance and the monitored safety properties.

The results show that an RL-based controller can perform the considered manipulation task in the simulated space environment under the stated assumptions. The optimized PPO agent yields a mean squared end-effector tracking error

of $K_2 = 0.0083$, a reduction of 67.7% relative to the default configuration ($K_2 = 0.0257$). The mean base orientation error decreases by a factor of approximately three (K_4 , $0.0667 \rightarrow 0.022$), and the mean base angular velocity is approximately halved (K_8 , $0.095 \text{ rad/s} \rightarrow 0.050 \text{ rad/s}$). No violations of the monitored safety properties were recorded over the 30 evaluation episodes ($K_5 = K_6 = 0$), compared with a 5.1% joint-limit violation rate for the default configuration. Control smoothness improves by 48.5% (K_7), and the energy consumption decreases by 16.0% (K_9). These effects are observed together in the reported evaluation rather than as an apparent trade-off between the selected objectives [1], [22].

An additional evaluation on a piecewise-linear end-effector reference assessed the transfer to a different trajectory. The optimized PPO agent performs better than the default configuration on most metrics. It increases the return K_1 from -0.4565 to -0.2977 and reduces the base orientation error K_4 by 44% and the energy consumption K_9 by 35%. This indicates a partial transfer of the learned policy. The control smoothness K_7 , however, is worse than for the default configuration. The hyperparameter configuration tuned for circular tracking therefore does not fully transfer to piecewise-linear references. This motivates either trajectory-diverse training or trajectory-specific tuning to improve the transfer across reference geometries [44].

Practical Implications. The performance of PPO and TRPO on this task indicates that policy-gradient methods with constrained updates are suitable candidates for continuous control tasks with strong dynamic coupling and safety constraints. The clipped surrogate objective of PPO provides bounded policy updates under the nonlinear free-floating dynamics. The runtime monitors (collision monitor and joint-limit monitor) supervise the training. They terminate episodes in which the policy would violate a monitored constraint and apply a terminal penalty [20], [24], [25]. Combined with the penalty-driven reward, this led to a final policy that did not violate the monitored constraints in the reported evaluation. The present simulation study does not

establish that this behavior transfers to a physical system. It suggests only that safety checks embedded in the training simulator may remain useful as runtime watchdogs, provided that the deployed system stays within the simulator's fidelity range, which remains to be validated [24], [31].

Limitations and Future Work. The study has several limitations. First, a sim-to-real gap remains. Beyond the idealizing assumptions stated in Section III (no sensor noise, no latency, rigid-body model, no external disturbances), real deployment introduces three further challenges. The first is a domain shift due to inaccurate mass and inertia parameters, joint friction, and link flexibility, which the URDF model does not capture. The second concerns onboard compute constraints. The learned policy must run in real time on embedded space-qualified hardware, typically ARM-based processors with limited floating-point throughput, whereas training was performed on a workstation CPU. The third is sensor-actuator latency. Delays between state measurement and torque application can destabilize a policy that was trained without latency. Mitigating these effects would require domain randomization over simulation parameters, latency injection during training, and ultimately HIL validation [24], [44]. Second, the RL agent exhibits hyperparameter sensitivity. The tuned PPO configuration is task-specific and may require re-tuning for different trajectories or mission scenarios. Third, the evaluation scope is limited. Each algorithm was trained in three runs and evaluated over 30 episodes from a fixed initial configuration, for a single robot model and a small set of reference trajectories. This does not fully characterize seed sensitivity, generalization across initial states, target trajectories, and robot parameters, or robustness to disturbances. The reported ranking and improvements are therefore conditional on the tested setup. Fourth, the state representation was manually designed. Learned representations or recurrent architectures (LSTM) might improve performance under partial observability. Fifth, the safety monitoring covers selected properties (collision events, joint limits, torque bound, and momentum consistency). It does not constitute a formal proof of closed-loop stability, task completion, or collision avoidance for the full nonlinear model, nor an exhaustive formal verification of the mission [23], [24]. Finally, no hardware testing was performed. Real-time computation limits, communication delays, and physical interactions remain to be validated on physical testbeds.

Future work could address these limitations in four directions. The first is domain randomization over simulation parameters and latency injection to improve the sim-to-real transfer [44] (see Fig. 19). The second is the extension to more complex tasks such as dual-arm coordination, on-orbit assembly, or active debris removal. The third is the use of recurrent policies, attention mechanisms, and safe RL algorithms with explicit constraint handling via Lagrange multipliers or barrier functions [24], [25], [31], [32]. The fourth is HIL validation on the authors' testbed for In-Space Robotized Services (IROS) with virtually simulated micro-

TABLE 13: Evaluation on a piecewise-linear end-effector trajectory (two-segment path). Values are averaged over $N_{\text{eval}} = 30$ episodes. Bold indicates the better value according to the KPI definition (K_1 higher is better, K_2 – K_9 lower is better).

Metric (KPI)	PPO (default)	PPO (optimized)
Mean Episodic Return (K_1)	-0.4565	-0.2977
Mean Squared EE Position Error (K_2)	0.0078	0.0070
Maximum EE Position Error (K_3)	0.2037	0.2036
Mean Base Orientation Error (K_4)	0.0412	0.0231
Collision Rate (K_5)	0	0
Joint Limit Violation Rate (K_6)	0	0
Control Smoothness (K_7 , Jerk)	0.5520	0.6839
Mean Base Angular Velocity (K_8)	0.0399	0.0339
Energy Consumption (K_9)	7.7144	5.0341

gravity and predicted energy consumption (see Fig. 19).

Conclusion. This work has presented a framework that combines deep reinforcement learning, multibody simulation, and formal-methods-based safety monitoring for Cartesian path tracking of a free-floating space robot. Six continuous-action RL algorithms were compared under unified conditions with the toolbox default hyperparameters. For the considered simulated task and evaluation setup, PPO showed the most favorable trade-off between tracking accuracy, base disturbance, and training stability. Subsequent hyperparameter tuning of the PPO agent yielded a policy that reduces the tracking error by 67.7% relative to the default configuration. It also reduces the base orientation error by a factor of approximately three and the energy consumption by 16.0%, and no joint-limit violations or collisions were observed in the 30 evaluation episodes. Runtime monitoring of selected safety properties in the training loop, together with an offline torque-bound check, helped to keep these results within the monitored constraints in the evaluated episodes. This addresses a part of the black-box nature of learned controllers. These simulation results, obtained under the stated idealizing assumptions, do not establish formal safety or operational readiness, but they support further investigation of deep RL with runtime safety monitoring for space manipulation tasks.

The results also illustrate the influence of algorithm choice and hyperparameter tuning on the learning performance for the considered task. They are consistent with the view that no single RL algorithm is uniformly best across applications. Under unified conditions, the choice of algorithm and hyperparameters visibly affected the achieved tracking accuracy and base stability. Systematic benchmarking of candidate algorithms is therefore a practical step in the selection of an appropriate RL controller for a given task.

Several open problems remain for real-world deployment, but the proposed framework provides a basis for further investigation. Possible application areas include satellite servicing, on-orbit assembly, and active debris removal, in which learning-based controllers could adapt to uncertain dynamics within predefined operational limits. One possible architecture for such applications combines an RL policy with a separate runtime layer that monitors the commands and overrides those that violate the specified constraints. In this combination, the learned policy provides adaptability, while the monitoring layer detects and intercepts violations of the specified properties [1], [25].

This work is one step in this direction. Future research at the intersection of robotics, reinforcement learning, and formal methods is required to extend the present results to physical deployment and to broader mission scenarios.

ACKNOWLEDGMENT

The authors used DeepL Translator/Write [45], [46] and OpenAI ChatGPT [47] for language editing and grammar enhancement. Anthropic Claude [48] was used as a coding assistant for developing and debugging the simulation and

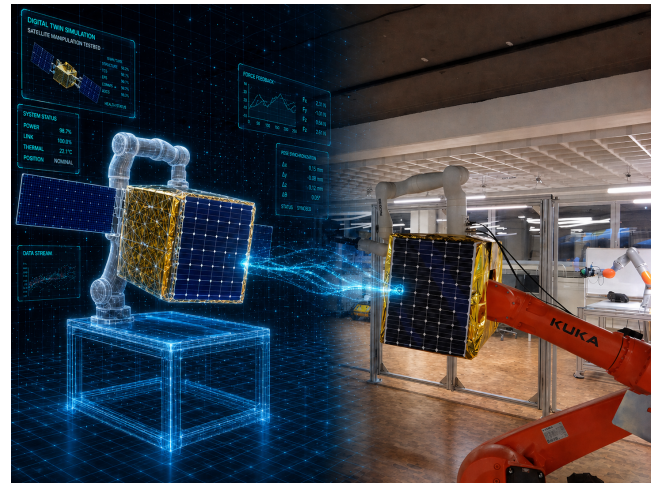


FIGURE 19: Gradual hybrid testbeds that integrate AI-driven digital twins (left) with physical robotic systems (right). Such testbeds allow policy optimization and domain-randomization strategies for sim-to-real transfer to be studied. Illustration created using a picture of the physical setup at Frankfurt University of Applied Sciences and with the help of GPT-Image-2.

evaluation scripts. All experiments, results, and conclusions were designed, checked, and verified by the authors.

REFERENCES

- [1] A. Flores-Abad, O. Ma, K. Pham, and S. Ulrich, "A review of space robotics technologies for on-orbit servicing," *Prog. Aerosp. Sci.*, vol. 68, pp. 1–26, 2014, doi: 10.1016/j.paerosci.2014.03.002.
- [2] K. Nanos and E. G. Papadopoulos, "On the dynamics and control of free-floating space manipulator systems in the presence of angular momentum," *Front. Robot. AI*, vol. 4, p. 26, 2017, doi: 10.3389/frobt.2017.00026.
- [3] E. Papadopoulos, I. Kaliakatsos, and D. Psarros, "Minimum fuel techniques for a space robot simulator with a reaction wheel and PWM thrusters," in *Proc. Eur. Control Conf. (ECC)*, 2007, pp. 3391–3398.
- [4] E. Papadopoulos and A. Abu-Abed, "Design and motion planning for a zero-reaction manipulator," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 1994, pp. 1554–1559, doi: 10.1109/ROBOT.1994.351367.
- [5] M. Mühlbauer, M. Chalon, M. Ulmer, and A. Albu-Schäffer, "Software for the SpaceDREAM robotic arm," in *Proc. IEEE Aerosp. Conf.*, 2025.
- [6] T. Rybus, "Robotic manipulators for in-orbit servicing and active debris removal: Review and comparison," *Prog. Aerosp. Sci.*, vol. 151, p. 104550, 2024.
- [7] E. Guiffo Kaigom, T. Jung, and J. Roßmann, "Optimal motion planning of a space robot with base disturbance minimization," in *Proc. 11th Symp. Adv. Space Technol. Robot. Autom.*, 2011.
- [8] E. Guiffo Kaigom, "Deep generative and discriminative digital twin endowed with variational autoencoder for unsupervised predictive thermal condition monitoring of physical robots in Industry 6.0 and Society 6.0," *IFAC-PapersOnLine*, vol. 59, pp. 7–12, 2025.
- [9] Z. Huang, G. Chen, Y. Shen, R. Wang, C. Liu, and L. Zhang, "An obstacle-avoidance motion planning method for redundant space robot via reinforcement learning," *Actuators*, vol. 12, no. 2, p. 69, 2023.
- [10] A. Al Ali and Z. Zhu, "Reinforcement learning for path planning of free-floating space robotic manipulator with collision avoidance and observation noise," *Front. Control Eng.*, vol. 5, p. 1394668, 2024.
- [11] M. D'Ambrosio, L. Capra, A. Brandonisio, and M. Lavagna, "Failure management via deep reinforcement learning for robotic in-orbit servicing," in *Proc. SPAICE—1st Joint ESA/IAA Conf. AI Space*, 2024, pp. 335–339.
- [12] A. Al Ali, J. Shi, and Z. Zhu, "Path planning of 6-DOF free-floating space robotic manipulators using reinforcement learning," *Acta Astronaut.*, vol. 224, pp. 367–378, 2024.

- [13] Y. Hu, D. Zhou, W. Yao, X. Shao, and G. Sun, "Deep reinforcement learning-based trajectory planning with continuous pose representation for 6-DoF free-floating space robot," *Aerosp. Sci. Technol.*, p. 110540, 2025.
- [14] O. Zhang, Z. Liu, X. Shao, W. Yao, L. Wu, and J. Liu, "Learning-based task space trajectory planning framework with pre-planning and post-processing for uncertain free-floating space robots," *IEEE Trans. Aerosp. Electron. Syst.*, early access, 2025.
- [15] Y. Wei, X. Bai, and H. Lu, "Trajectory planning of free-floating space robot for non-cooperative tumbling target capture based on deep reinforcement learning," *Robotica*, vol. 43, pp. 2674–2692, 2025.
- [16] Y. Wu, Z. Yu, C. Li, M. He, B. Hua, and Z. Chen, "Reinforcement learning in dual-arm trajectory planning for a free-floating space robot," *Aerosp. Sci. Technol.*, vol. 98, p. 105657, 2020.
- [17] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 1889–1897.
- [18] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1587–1596.
- [19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1861–1870.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
- [21] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, pp. 229–256, 1992.
- [22] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [23] MathWorks, "Simulink Design Verifier," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sldv/>
- [24] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *J. Mach. Learn. Res.*, vol. 16, pp. 1437–1480, 2015.
- [25] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2669–2678.
- [26] Open Robotics, "URDF (Unified Robot Description Format)," ROS 2 Documentation, 2024. [Online]. Available: <https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html>
- [27] MathWorks, "Import URDF models," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sm/ug/urdf-import.html>
- [28] MathWorks, "smimport—Import a CAD, URDF, or Robotics System Toolbox model," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sm/ref/smimport.html>
- [29] MathWorks, "Mechanism Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sm/ref/mechanismconfiguration.html>
- [30] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [31] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 22–31.
- [32] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. Eur. Control Conf. (ECC)*, 2019, pp. 3420–3431.
- [33] F. Leutert et al., "AI-enabled cyber-physical in-orbit factory—AI approaches based on digital twin technology for robotic small satellite production," *Acta Astronaut.*, vol. 217, pp. 1–17, 2024.
- [34] M. Mühlbauer et al., "Towards an in-orbit cubesat factory," in *Proc. 76th Int. Astronaut. Congr. (IAC)*, 2025.
- [35] O. Maqbool and J. Roßmann, "Systems aware scenario engineering for life cycle-spanning verification and validation of complex CPS," *Authorea Preprints*, 2025. [Online]. Available: <https://www.authorea.com>
- [36] J. Virgili-Llop, "SPART: Spacecraft Robotics Toolkit," 2017. [Online]. Available: <https://github.com/NPS-SRL/SPART>
- [37] E. Ribeiro, R. Mendes, M. Terra, and V. Grassi, "Dynamic parameter identification of a 7-DoF lightweight robot manipulator using probabilistic differential optimization," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, 2022, pp. 1917–1923.
- [38] MathWorks, "6-DOF Joint (Simscape Multibody)," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sm/ref/6dofjoint.html>
- [39] MathWorks, "Spatial Contact Force," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/sm/ref/spatialcontactforce.html>
- [40] MathWorks, "Solver Configuration," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/simscape/ref/solverconfiguration.html>
- [41] MathWorks, "Fixed-step size (fundamental sample time)," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/simulink/gui/fixedstepsizefundamentalsampletime.html>
- [42] MathWorks, "Reinforcement Learning Toolbox—rlPPOAgent," MATLAB & Simulink Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rlppoagent.html>
- [43] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [44] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RISJ Int. Conf. Intell. Robots Syst. (IROS)*, 2017, pp. 23–30.
- [45] DeepL SE, "DeepL Translator," [Online]. Available: <https://www.deepl.com/de/translator>
- [46] DeepL SE, "DeepL Write," [Online]. Available: <https://www.deepl.com/de/write>
- [47] OpenAI, "ChatGPT," [Online]. Available: <https://chat.openai.com>
- [48] Anthropic, "Claude," [Online]. Available: <https://claude.ai>
- [49] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RISJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
- [50] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RISJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, 2004, pp. 2149–2154.
- [51] M. Körber, J. Lange, S. Rediske, S. Steinmann, and R. Glück, "Comparing popular simulation environments in the scope of robotics and reinforcement learning," *arXiv:2103.04616*, 2021.
- [52] J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A review of physics simulators for robotic applications," *IEEE Access*, vol. 9, pp. 51416–51431, 2021.



PATRICK S. ERMISCH Patrick S. Ermisch was born in Frankfurt am Main, Germany, on December 18, 2002. He received the Abitur degree and is currently pursuing the B.Eng. degree in mechatronics at Frankfurt University of Applied Sciences, Frankfurt, Germany, with an expected graduation in 2026. His major field of study is mechatronics.

He has gained practical experience through an internship at SEW-EURODRIVE, where he contributed to the further development of a mobile application for the manual control of logistics robots. In addition, he worked as a tutor for Physics I and Physics II and as a student research assistant, supporting research on particle simulation and the development of an application for visualization and simulation purposes. His research interests include reinforcement learning for robotics, space robotic systems, and simulation-based control methods.

Mr. Ermisch has been listed on the Dean's List of Frankfurt University of Applied Sciences multiple times in recognition of his academic performance.



ERIC GUIFFO KAIGOM was born in Douala, Cameroon. He received the Dipl.-Ing. and Dr.-Ing. (summa cum laude) degrees in electrical engineering from RWTH Aachen University, Germany. He was a Research Associate with the Institute for Man-Machine Interaction at RWTH Aachen University, where he led the “Intelligent Robots” team and coordinated technology development and transfer activities with partners in robotics-based manufacturing and on-orbit servicing, including projects funded under the European Union’s Horizon 2020 and In-Space Automation programs. He then worked in industry as Head of Emerging Technologies. Dr. Kaigom is currently a Full Professor with Frankfurt University of Applied Sciences, Germany. His research interests include robotics, digital twins, and artificial intelligence / machine learning.

...